

Algorithm Engineering

Esercizi di Teoria — testi e risoluzioni dettagliate

Tutte le richieste di teoria di appelli, midterm, final term e orali 2020–2025

Versione riveduta e completata — luglio 2026

Come usare questo documento

Parte I: i testi delle domande di teoria, organizzati per argomento. Ogni testo ha un identificatore (es. [5.1]) e accanto sono indicate le prove d'esame in cui è uscito. **Parte II:** le risoluzioni, con dimostrazioni complete e commenti, sviluppate a partire dalla dispensa del corso e da *Pearls of Algorithm Engineering* (Ferragina, 2023). La numerazione delle risoluzioni rispecchia quella dei testi.

Copertura. Questa versione è stata verificata contro *tutte* le prove presenti nella cartella **esercizi/** (35 prove con testo leggibile, dal 10/01/2020 al 15/07/2025, inclusi i .doc del 2020 e del 2022 e l'orale del 25/07/2022). Rispetto alla versione precedente sono stati aggiunti gli item mancanti: [1.5] sorting vs permuting nel modello RAM e I/O (orale 25/07/22); [3.3] sampling con n noto e prova di correttezza (midterm 06/11/23); [5.9] prova che l'altezza media del Treap è $O(\log n)$ (07/02/20, orale 25/07/22); [7.7] algoritmo lineare di Kasai per l'array LCP (orale 25/07/22, domanda “da lode”); [8.3] come ricavare n dagli array L, H di Elias–Fano (orale 25/07/22); [9.3] bound sull'errore di troncamento di un numero binario (10/01/20); e la nuova sezione 11 con le tre *domande di progettazione* (07/02/20, 02/02/22, 11/11/24). Le convenzioni (bit alti di Elias–Fano, tabelle FC/SYMB, Wheeler code) sono state allineate a quelle effettivamente usate nei compiti del docente.

Legenda. Nella Parte II ogni risoluzione è preceduta dal *testo dell'esercizio* (riquadro azzurro) con le date in cui è uscito; i passaggi e i risultati da ricordare sono **evidenziati in rosso** o incorniciati. Il simbolo ★ marca gli argomenti usciti in *tre o più* prove: da sapere alla perfezione.

Nota sulle soluzioni ufficiali. Le soluzioni ufficiali esistono solo come scansioni manoscritte della parte di esercizi pratici; per la teoria non esistono soluzioni ufficiali in formato testo, quindi le risoluzioni qui riportate sono ricostruite dalle fonti del corso.

Tabella di copertura: prova per prova

Data	Tipo	Domande di teoria e riferimenti
10/01/20	appello	cammini cuckoo c^{-L}/r [5.3]; spazio Elias–Fano [8.1]; errore di troncamento binario [9.3]
07/02/20	appello	design I/O max-satellite [11.1]; bound sorting D dischi con prova e confronto con l'ottimo [1.3][1.4]; profondità media Treap [5.9]
12/06/20	appello	suffix tree: sottostringa con $\geq L$ occorrenze [7.6]
10/07/20	appello	compacted trie con fan-out hash per ordinare stringhe [6.5]
11/11/20	midterm	Snow Plow con prob. heap $3/4$ [2.1]; collisioni attese con hash universale [5.2]; Bloom: $p_0 = \frac{1}{2}$ con k ottimo [5.4]
07/09/21	appello	costruzione SA via qsort, e caso LCP $\leq c$ [7.3]; perché l'aritmetico parte dall'elemento centrale [9.6]
14/01/22	final term	intersezione di liste Elias–Fano con Access/GEQ [4.4]

Data	Tipo	Domande di teoria e riferimenti
02/02/22	appello	design: finestra di k posizioni con q occorrenze via SA [11.2]
25/07/22	orale	sorting/permuting in RAM e I/O [1.5]; lower bound stringhe + Multikey QS [2.2][2.3]; Treap: definizione e altezza media [5.8][5.9]; ricavare n da Elias–Fano [8.3]; LCP in tempo lineare (lode) [7.7]
16/01/23	final term	LSD RadixSort [2.4]; hash universale [5.1]; Rank [10.1]; def. SA+LCP [7.1]
10/02/23	appello	teorema cuckoo [5.3]; Multikey QS + lower bound [2.2][2.3]; upper bound I/O sort/permute/striping [1.2]; Treap: definizione e split [5.8]
05/06/23	appello	lower bound + Multikey QS + RadixSort ottimo [2.2][2.3][2.4]; Rank + spazio [10.1]; Bloom su 2 server [5.6]
24/07/23	appello	SA via qsort e I/O [7.3]; ricerca lessicografica con Patricia [6.1]; Bloom: definizione + falso positivo [5.5]
07/09/23	appello	Multikey QS [2.3]; hash universale [5.1]; SA/LCP + ricerca + prova [7.2]
02/11/23	appello	Reservoir + correttezza [3.2]; hash universale [5.1]; Rank [10.1]; def. SA+LCP [7.1]
06/11/23	midterm	sampling con n noto + correttezza ($m = 1$) [3.3]; universale + chaining $O(1)$ [5.2]
12/12/23	final term	upper bound Arithmetic [9.2]; spazio della struttura di Rank [10.2]
15/01/24	appello	Snow Plow $2M$ e $p = \frac{1}{4}$ [2.1]; COUNT con SA + retrieval su disco [7.4]; doubling [4.3]
09/02/24	appello	MergeSort multi-way I/O + disk striping [1.1][1.3]; Canonical Huffman [9.1]; intersezione two-level [4.1]
27/05/24	appello	Bloom [5.4]; spazio Elias–Fano [8.1]; MergeSort I/O [1.1]; storage di un dizionario di stringhe [6.4]
21/06/24	appello	Snow Plow con $p = \frac{1}{8}$ [2.1]; two-level scan + prova [4.1]; Canonical Huffman [9.1]
11/07/24	appello	Reservoir ($m < n/2$) + correttezza [3.2]; LSD su b bit [2.4]; mutual partition [4.2]; Access su Elias–Fano [8.2]
11/11/24	midterm	altezza Skip List [5.7]; errore del Bloom [5.4]; two-level intersection [4.1]; design range di frequenze [11.3]
09/12/24	final term	upper bound Arithmetic [9.2]
10/01/25	appello	sorting con D dischi vs ottimo [1.4]; Bounded QuickSort [2.5]; hash universale [5.1]; LSD + correttezza [2.4]; Rank $O(1)/o(m)$ [10.2]
04/02/25	appello	MergeSort I/O con 1 e D dischi [1.1][1.3]; Bounded QuickSort [2.5]; Skip List [5.7]; Rank [10.2]; spazio Elias–Fano [8.1]
06/03/25	appello	pseudocodice Reservoir [3.1]; bit di Huffman vs Arithmetic + quando [9.4]; Compacted vs Patricia [6.2]
03/06/25	appello	pseudocodice Reservoir [3.1]; Huffman vs Arithmetic [9.4]; teorema cuckoo [5.3]; disk striping [1.3]; Compacted vs Patricia nel two-level [6.3]
23/06/25	appello	LSD + correttezza [2.4]; Bounded QuickSort [2.5]; definizione Treap [5.8]; invarianza per permutazione dell’Arithmetic [9.5]
15/07/25	appello	errore del Bloom [5.4]; two-level intersection [4.1]; ricerca di P in SA [7.5]; Canonical Huffman [9.1]

Prove *senza* domande di teoria (solo esercizi di simulazione): 03/09/20, 16/12/20, 20/01/21, 10/02/21 (a parte il commento di complessità del COUNT, coperto da [7.4]), 09/04/21, 08/06/21, 28/10/21, 08/11/21, 15/12/21, 11/04/22, 13/06/22, 04/07/22, 25/07/22 (scritto).

Indice

I	Testi degli esercizi di teoria	4
1	Modelli e complessità I/O	4
2	Ordinamento	4
3	Random Sampling	4
4	Set Intersection	5
5	Hashing, Bloom Filter, Skip List, Treap	5
6	Prefix Search	6
7	Substring Search	6
8	Integer Coding	6
9	Statistical Coding	7
10	Strutture dati compresse	7
11	Domande di progettazione	7
II	Risoluzioni dettagliate	8
12	Modelli e complessità I/O — risoluzioni	8
13	Ordinamento — risoluzioni	10
14	Random Sampling — risoluzioni	14
15	Set Intersection — risoluzioni	15
16	Hashing, Bloom Filter, Skip List, Treap — risoluzioni	17
17	Prefix Search — risoluzioni	22
18	Substring Search — risoluzioni	24
19	Integer Coding — risoluzioni	26

20 Statistical Coding — risoluzioni	29
21 Strutture dati compresse — risoluzioni	31
22 Domande di progettazione — risoluzioni	33

Parte I

Testi degli esercizi di teoria

1 Modelli e complessità I/O

- [1.1] ★ Enuncia la complessità I/O del MergeSort multi-way in funzione di M , B , N . (09/02/24, 27/05/24, 04/02/25)
- [1.2] Enuncia gli upper bound per: ordinare n item atomici in un modello a due livelli con memoria interna M , pagina B e 1 disco; permutarli nello stesso modello con 1 disco; ordinarli con D dischi usando il disk striping. (10/02/23)
- [1.3] ★ Descrivi la tecnica del disk striping ed enuncia il bound I/O del MergeSort quando lo si usa per ordinare n item in una memoria a due livelli con D dischi, memoria M , pagina B . (07/02/20 con prova, 09/02/24, 04/02/25, 03/06/25)
- [1.4] Enuncia la complessità I/O di ordinare n item con memoria M , pagina B e D dischi; confrontala con il bound ottimo e discuti le differenze. (07/02/20, 10/01/25)
- [1.5] Dati n item atomici, scrivi la complessità in tempo dei problemi di sorting e di permuting nel modello RAM, poi la loro complessità I/O nel modello a due livelli (pagina B , memoria M); commenta se i due problemi sono computazionalmente equivalenti in senso asintotico e per quali valori dei parametri. (orale 25/07/22)

2 Ordinamento

- [2.1] ★ Dimostra che la lunghezza attesa della sequenza ordinata (run) prodotta da Snow Plow è $2M$. Varianti chieste: quanto vale se la probabilità di finire nell'*unsorted bucket* è $\frac{1}{4}$ invece di $\frac{1}{2}$? (15/01/24) Se è $\frac{1}{8}$? (21/06/24) Se la probabilità di andare nello *heap* è $\frac{3}{4}$? (11/11/20)
- [2.2] ★ Enuncia e dimostra il lower bound (comparison-based) al tempo per ordinare n stringhe di lunghezza totale N su alfabeto di taglia σ . (10/02/23, 05/06/23, orale 25/07/22)
- [2.3] ★ Scrivi lo pseudocodice del Multikey QuickSort su n stringhe di lunghezza totale N ; enuncia e dimostra la sua complessità temporale e mostra che è time-optimal rispetto al lower bound di [2.2]. (10/02/23, 05/06/23, 07/09/23, orale 25/07/22)
- [2.4] ★ Abbozza l'algoritmo LSD RadixSort per n chiavi di b bit ciascuna; enuncia e dimostra la sua complessità temporale; dimostra la correttezza dell'ordinamento prodotto (caso stringhe binarie). Variante: complessità del RadixSort ottimo su n stringhe binarie di lunghezza totale N . (16/01/23, 05/06/23, 11/07/24, 10/01/25, 23/06/25)
- [2.5] ★ Scrivi e commenta lo pseudocodice del Bounded QuickSort, cioè la variante che garantisce uno stack di taglia $O(\log n)$ (equivalentemente: $O(\log n)$ chiamate ricorsive annidate per qualunque input). (10/01/25, 04/02/25, 23/06/25)

3 Random Sampling

- [3.1] ★ Scrivi (solo) lo pseudocodice del Reservoir Sampling. (06/03/25, 03/06/25)
- [3.2] ★ Data una sequenza S di n item (n ignoto) e un intero m (eventualmente $m < n/2$): mostra lo pseudocodice del Reservoir Sampling e dimostrane la correttezza, cioè che ogni item è campionato con probabilità m/n (campionamento uniforme). (02/11/23, 11/07/24)

- [3.3] Data una sequenza S di n item con n noto e un intero m , scrivi lo pseudocodice dell'algoritmo di sampling che estrae m item uniformemente a caso, e dimostrane la correttezza (probabilità m/n) nel caso $m = 1$. (06/11/23; simulazioni in 13/06/22, 10/02/23, 07/09/23)

4 Set Intersection

- [4.1] ★Date due liste ordinate L_1, L_2 di lunghezze n e m : descrivi l'algoritmo *two-level* (detto anche two-level scan / two-level memory / two-level storage) per calcolarne l'intersezione; enuncia e dimostra la sua complessità. (09/02/24, 21/06/24, 11/11/24, 15/07/25)
- [4.2] Descrivi l'algoritmo *mutual partition* per l'intersezione di L_1, L_2 , con la sua complessità. (11/07/24)
- [4.3] Descrivi il *doubling algorithm* (binary search con salti esponenziali, "galloping") per l'intersezione; enuncia e dimostra la sua complessità. (15/01/24)
- [4.4] Scrivi lo pseudocodice per intersecare due liste ordinate L_1, L_2 (di n_1 e n_2 elementi) codificate con Elias-Fano, usando come primitive $\text{Access}(L, i)$ e $\text{GEQ}(L, x)$; valuta e commenta la complessità. (14/01/22)

5 Hashing, Bloom Filter, Skip List, Treap

- [5.1] ★Definisci una classe di funzioni hash universali; fornisci un esempio e dimostrane l'universalità. (16/01/23, 07/09/23, 02/11/23, 10/01/25)
- [5.2] Definisci una classe universale e mostra che, usata nell'hashing con chaining, ottiene lunghezza attesa $O(1)$ per catena quando $m = O(n)$; da qui il calcolo del numero atteso di collisioni. (06/11/23; esercizi collegati in 10/07/20 e 11/11/20)
- [5.3] ★Enuncia e dimostra il teorema sul cuckoo graph: per ogni coppia di posizioni i, j della tabella cuckoo, la probabilità che il cammino minimo $i \rightarrow j$ abbia lunghezza L è al più c^{-L}/r , dove r è il numero di posizioni della tabella, n le chiavi memorizzate e $r \geq 2cn$. Commenta l'impatto sul load factor. (10/01/20, 10/02/23, 03/06/25)
- [5.4] ★Calcola la probabilità che una posizione del bit-array del Bloom Filter (taglia m , n chiavi inserite) sia 0, e deriva/dimostra la probabilità d'errore complessiva. Variante: dimostra che tale probabilità è $\frac{1}{2}$ quando il numero di funzioni hash è quello ottimo. (11/11/20, 27/05/24, 11/11/24, 15/07/25)
- [5.5] ★Definisci un Bloom Filter su un dizionario di n chiavi e le operazioni supportate; enuncia e dimostra il risultato sul falso positivo. (24/07/23)
- [5.6] Dati due insiemi A, B memorizzati su due server, mostra come calcolarne l'intersezione usando un Bloom Filter e un solo round di comunicazione, ammettendo errori. (05/06/23)
- [5.7] Dimostra che l'altezza (profondità) media di una Skip List con n nodi è $O(\log n)$. (11/11/24, 04/02/25)
- [5.8] ★Definisci la struttura Treap su n coppie $\langle \text{chiave}, \text{priorità} \rangle$ con priorità minima nella radice; descrivi le operazioni supportate e come si implementa lo split su una chiave K non presente. (10/02/23, 23/06/25, orale 25/07/22)
- [5.9] Enuncia e dimostra che la profondità media di un Treap con priorità scelte uniformemente a caso è logaritmica nel numero di chiavi. (07/02/20, orale 25/07/22)

6 Prefix Search

- [6.1] Dato un Patricia Trie su un dizionario S di n stringhe, descrivi come cercare la posizione lessicografica di $P[1, p]$ in S ; enuncia la complessità in tempo e in I/O. (24/07/23)
- [6.2] Descrivi la differenza fra Compacted Trie e Patricia Trie in termini di spazio e tempo di query, in funzione di n , della lunghezza totale N e della lunghezza del pattern P . (06/03/25)
- [6.3] Qual è la differenza principale fra Compacted Trie e Patricia Trie, e come è stata sfruttata per costruire un indice *two-level* efficiente per un insieme di stringhe? (03/06/25)
- [6.4] Dato un dizionario D di n stringhe di lunghezza variabile e lunghezza totale N , discuti almeno 3 soluzioni di memorizzazione, commentando spazio e costo tempo/I/O di $\text{Access}(i)$ (recupero della i -esima stringa). (27/05/24)
- [6.5] Descrivi come usare un compacted trie (con fan-out dei nodi in hash table) per ordinare n stringhe di lunghezza totale L ; commenta tempo e spazio. (10/07/20)

7 Substring Search

- [7.1] Definisci formalmente il suffix array SA di un testo $T[1, n]$ e il corrispondente array LCP. (16/01/23, 02/11/23)
- [7.2] ★Come [7.1], e inoltre: descrivi come cercare $P[1, p]$ in SA ; enuncia e dimostra la complessità della ricerca. (07/09/23)
- [7.3] Definisci SA ; scrivi lo pseudocodice per costruirlo via qsort; enuncia e commenta la complessità worst-case (tempo e I/O) di questa costruzione, e il caso in cui il massimo LCP fra i suffissi è limitato da una costante c . (07/09/21, 24/07/23)
- [7.4] ★Mostra come CONTARE in $T[1, n]$ tutte le occorrenze di $P[1, p]$ con un Suffix Array in memoria; enuncia e dimostra la complessità; qual è il costo I/O del RETRIEVAL delle posizioni se SA è su disco? (10/02/21, 15/01/24)
- [7.5] ★Descrivi come cercare $P[1, p]$ in un SA costruito su $T[1, n]$ e commenta la complessità in funzione di p e n . (15/07/25)
- [7.6] Dato il suffix tree di $T[1, n]$, come si calcola la sottostringa più lunga che occorre almeno L volte? Complessità in funzione di n . (12/06/20)
- [7.7] Scrivi lo pseudocodice dell'algoritmo lineare (Kasai et al.) che calcola l'array LCP dato T e SA , e dimostra che è $O(n)$. (orale 25/07/22, domanda "da lode"; simulazioni in 11/11/24, 06/03/25, 15/07/25)

8 Integer Coding

- [8.1] ★Data una sequenza monotona di n interi non negativi $< u$, enuncia e dimostra lo spazio (in bit) della codifica di Elias-Fano. (10/01/20, 27/05/24, 04/02/25)
- [8.2] Scrivi l'algoritmo per eseguire $\text{Access}(i)$ su una sequenza codificata con Elias-Fano; cenno a $\text{NextGEQ}(x)$. (11/07/24; simulazioni in molti appelli)
- [8.3] Dati gli array L e H di una codifica Elias-Fano, mostra come ricavare il valore di n (numero di interi codificati), motivando la risposta. (orale 25/07/22; è il "hint" ricorrente degli esercizi: 11/11/20, 12/12/23, 09/12/24, 06/03/25, 23/06/25)
- [8.4] (Riferimento per gli esercizi di calcolo) Numero di bit emessi dai codici unario, γ , δ , Rice e (s, c) -dense su un intero x . (vari appelli)

9 Statistical Coding

- [9.1] ★Descrivi perché è stato introdotto il Canonical Huffman; specifica quali strutture dati tiene nel preambolo del file compresso; scrivi lo pseudocodice per decomprimere un simbolo. (09/02/24, 21/06/24, 15/07/25)
- [9.2] Dimostra l'upper bound in bit della codifica aritmetica, in funzione dell'entropia e della lunghezza del testo in input. (12/12/23, 09/12/24)
- [9.3] Dimostra un upper bound alla variazione indotta dal troncamento a k cifre di un numero < 1 rappresentato in binario come $.b_1b_2b_3\dots$ (10/01/20)
- [9.4] Enuncia il numero di bit emessi da Huffman e da Arithmetic dato un testo T di lunghezza n ed entropia empirica H ; commenta quando conviene comprimere con l'uno o con l'altro, tenendo conto anche dell'efficienza in tempo. (06/03/25, 03/06/25)
- [9.5] Dimostra che due testi T_1, T_2 , l'uno permutazione dell'altro, sono compressi con lo stesso numero di bit dalla codifica aritmetica. (23/06/25)
- [9.6] Perché la codifica aritmetica non emette l'estremo sinistro dell'intervallo finale, ma parte dall'elemento centrale e lo tronca? (07/09/21)

10 Strutture dati compresse

- [10.1] ★Descrivi la struttura dati per supportare **Rank** su un array binario $B[1, n]$ in tempo $O(1)$ e valutarne lo spazio in bit. (16/01/23, 05/06/23, 02/11/23)
- [10.2] ★Dimostra la complessità in tempo e in spazio (in bit) della soluzione che implementa **Rank** in $O(1)$ e $o(m)$ bit su $B[1, m]$. (12/12/23, 10/01/25, 04/02/25)
- [10.3] (Riferimento) Codifica succinta di alberi binari in $B[1, 2n + 1]$ bit e navigazione con Rank/Select; LOUDS per alberi generali. (base degli esercizi: 11/11/20, 15/12/21, 25/07/22, 16/01/23, 10/02/23, 24/07/23, 12/12/23, 09/02/24, 10/01/25, 04/02/25, 06/03/25, 03/06/25, 23/06/25)

11 Domande di progettazione

- [11.1] È data una sequenza S di n coppie $\langle \text{stringa}, \text{occ} \rangle$, dove più coppie possono riferirsi alla stessa stringa e occ è un valore satellite intero; sia s il numero di stringhe distinte. Progetta un algoritmo I/O-efficiente che calcola la stringa con massima somma dei valori satellite, valutandone la complessità I/O nei tre casi: (a) M molto più piccola di s ; (b) $M = O(s \log n)$ bit ammettendo errori in probabilità; (c) M può contenere $O(s)$ puntatori a stringhe e $O(s)$ caratteri e contatori (hint: soluzione trie-based, ma non un trie classico). (07/02/20)
- [11.2] Dato un testo $T[1, n]$ indicizzato da un Suffix Array, progetta un algoritmo che, dati $P[1, p]$ e due interi positivi k e q , stabilisce se esiste un range di k posizioni contigue $T[i, i + k - 1]$ in cui *iniziano* almeno q occorrenze di P ; stampa i se esiste, altrimenti -1 . (02/02/22)
- [11.3] È dato un insieme di coppie $\langle \text{frequenza}, \text{parola} \rangle$. Progetta una struttura dati e un algoritmo che, dati un range di frequenze $[f_{\min}, f_{\max}]$ e una parola W , restituiscono tutte le parole lessicograficamente maggiori di W con frequenza nel range; discuti complessità in tempo e spazio. (11/11/24)

Parte II

Risoluzioni dettagliate

12 Modelli e complessità I/O — risoluzioni

[1.1] ★ MergeSort multi-way: complessità I/O

Testo dell'esercizio. Enuncia la complessità I/O del MergeSort multi-way in funzione di M , B , N .
(09/02/24, 27/05/24, 04/02/25)

Enunciato. Con memoria interna M , pagina di disco B e N item da ordinare:

$$\text{MergeSort multi-way} = O\left(\frac{N}{B} \log_{M/B} \frac{N}{M}\right) \text{ I/O}$$

che coincide con il bound ottimo $\text{sort}(N) = \Theta\left(\frac{N}{B} \log_{M/B} \frac{N}{M}\right)$.

Da dove viene la formula (serve saperla giustificare). L'algoritmo ha due fasi.

Fase 1 — formazione dei run. Si legge un blocco di M item alla volta, lo si ordina in memoria interna (con qualunque algoritmo ottimo, oppure con QuickSort/HeapSort) e lo si riscrive su disco come run ordinato. Ogni item è letto e scritto una volta: $O(N/B)$ I/O. Alla fine ci sono N/M run ordinati di lunghezza M .

Fase 2 — merge a k vie. La memoria può tenere M/B pagine. Si fondono $k = \frac{M}{B} - 1$ run alla volta: una pagina di buffer per ciascuno dei k run in ingresso più una pagina di output (quando un buffer di input si svuota si carica la pagina successiva del suo run; quando l'output si riempie si scarica su disco). Il minimo corrente fra i k run si trova con un min-heap di k elementi ($O(\log k)$ tempo per estrazione, ma qui contiamo solo gli I/O). Ogni passata di merge legge e scrive tutti gli N item ($O(N/B)$ I/O) e riduce il numero di run di un fattore $k = \Theta(M/B)$. Il numero di passate per passare da N/M run a 1 è dunque

$$\log_k \frac{N}{M} = \log_{M/B} \frac{N}{M}.$$

Totale: $O\left(\frac{N}{B}\right)$ (fase 1) + $O\left(\frac{N}{B} \log_{M/B} \frac{N}{M}\right)$ (fase 2), cioè la formula enunciata. **Il punto qualificante è la base del logaritmo: M/B invece di 2** (del MergeSort binario). Con valori realistici il fattore $\log_{M/B}(N/M)$ vale 1 o 2: in pratica “due passate sul disco”.

[1.2] Upper bound: sorting, permuting, striping

Testo dell'esercizio. Enuncia gli upper bound per: ordinare n item atomici in un modello a due livelli con memoria interna M , pagina B e 1 disco; permutarli nello stesso modello con 1 disco; ordinarli con D dischi usando il disk striping. (10/02/23)

Sorting, 1 disco. $O\left(\frac{n}{B} \log_{M/B} \frac{n}{M}\right)$ I/O (MergeSort multi-way, [1.1]).

Permuting, 1 disco.

$$O(\min\{n, \text{sort}(n)\}) \text{ I/O}$$

Le due alternative: (i) muovere gli item uno alla volta nella posizione finale, $O(1)$ I/O ciascuno, totale $O(n)$; (ii) attaccare a ogni item la sua posizione di destinazione come chiave e ordinare con

MergeSort multi-way, totale $O(\text{sort}(n))$. Si sceglie il minimo. (Il lower bound corrispondente è $\Omega\left(\min\left\{n, \frac{n}{B} \log_{M/B} \frac{n}{B}\right\}\right)$, e per i valori realistici dei parametri i due problemi costano uguale: si veda [1.5].)

Sorting, **D dischi** con disk striping. $O\left(\frac{n}{DB} \log_{M/(DB)} \frac{n}{M}\right)$ I/O: si veda [1.3].

[1.3] ★ Disk striping e MergeSort su D dischi (con prova)

Testo dell'esercizio. Descrivi la tecnica del disk striping ed enuncia il bound I/O del MergeSort quando lo si usa per ordinare n item in una memoria a due livelli con D dischi, memoria M , pagina B . (07/02/20 con prova, 09/02/24, 04/02/25, 03/06/25)

La tecnica. Il disk striping fa lavorare D dischi come **un unico disco logico con pagine di taglia $D \cdot B$** . La pagina logica i è composta dalle D pagine fisiche di indice i , una per disco (la “striscia” i -esima): i primi B item sulla pagina i del disco 1, i successivi B sulla pagina i del disco 2, ecc. Poiché i D dischi possono **leggere/scrivere in parallelo**, un singolo I/O logico trasferisce $D \cdot B$ item al costo (in tempo) **di un I/O fisico**. Il vantaggio è la semplicità: qualunque algoritmo pensato per 1 disco funziona immutato.

Bound (prova). Si esegue il MergeSort multi-way di [1.1] sul disco logico, cioè **sostituendo $B \rightarrow DB$ nell'analisi**. Contando gli I/O logici:

$$O\left(\frac{n}{DB} \log_{M/(DB)} \frac{n}{M}\right) \text{ I/O}$$

Ogni I/O logico corrisponde a D I/O fisici *simultanei*, quindi questa è anche la misura del tempo di I/O con D dischi in parallelo.

[1.4] Confronto con il bound ottimo su D dischi

Testo dell'esercizio. Enuncia la complessità I/O di ordinare n item con memoria M , pagina B e D dischi; confrontala con il bound ottimo e discuti le differenze. (07/02/20, 10/01/25)

Il bound ottimo per **ordinare** con D dischi è

$$\Theta\left(\frac{n}{DB} \log_{M/B} \frac{n}{DB}\right) \text{ I/O}$$

raggiunto da algoritmi più sofisticati (es. Greed Sort), che tengono la base del logaritmo a M/B pur dividendo il volume di dati fra i dischi.

Differenze. Il disk striping ([1.3]) ottiene lo stesso fattore $\frac{n}{DB}$ davanti al logaritmo, ma **la base del logaritmo peggiora da M/B a $M/(DB)$** : usando pagine logiche di taglia DB , il fan-out del merge scende da $\Theta(M/B)$ a $\Theta(M/(DB))$, cioè **si fondono meno run per passata e servono più passate**. Il **rapporto** fra i due bound è

$$\frac{\log_{M/(DB)}(n/M)}{\log_{M/B}(n/DB)} \approx \frac{\log(M/B)}{\log(M/(DB))},$$

che vale ≈ 1 quando $DB \ll M$ (**pochi dischi: striping quasi ottimo**) e **diverge** quando $DB \rightarrow M$ (con D grande il merge diventa quasi binario: **striping non ottimale per D grande**). In pratica si usa comunque lo striping per la sua semplicità.

[1.5] Sorting vs permuting: RAM vs modello a due livelli

Testo dell'esercizio. Dati n item atomici, scrivi la complessità in tempo dei problemi di sorting e di permuting nel modello RAM, poi la loro complessità I/O nel modello a due livelli (pagina B , memoria M); commenta se i due problemi sono computazionalmente equivalenti in senso asintotico e per quali valori dei parametri. (orale 25/07/22)

Modello RAM. Sorting (comparison-based): $\Theta(n \log n)$. Permuting: $\Theta(n)$ (si muove ogni item direttamente in posizione finale). In RAM permutare è strettamente più facile di un fattore $\log n$.

Modello a due livelli. Sorting: $\Theta\left(\frac{n}{B} \log_{M/B} \frac{n}{M}\right)$ I/O. Permuting: $\Theta\left(\min\left\{n, \frac{n}{B} \log_{M/B} \frac{n}{B}\right\}\right)$ I/O.

Quando coincidono. Il permuting costa quanto il sorting a meno che il termine “naive” n non sia il minimo, cioè a meno che $B < \log_{M/B} \frac{n}{B}$. Con i valori reali dei parametri (B dell'ordine di 10^3 – 10^4 item, mentre $\log_{M/B}(n/B)$ vale pochissime unità anche per dataset enormi) risulta sempre $B \gg \log_{M/B}(n/B)$: quindi nel modello a due livelli sorting e permuting costano asintoticamente uguale. È la differenza concettuale più importante col modello RAM: su disco non si riesce a sfruttare la conoscenza della permutazione finale, perché muovere item singoli paga comunque un I/O intero a spostamento; tanto vale ordinare. Morale del corso: in memoria esterna anche i problemi “facili” costano come il sorting, e $\text{sort}(n)$ è l'unità di misura di riferimento.

13 Ordinamento — risoluzioni

[2.1] ★ Snow Plow: lunghezza attesa del run è $2M$

Testo dell'esercizio. Dimostra che la lunghezza attesa della sequenza ordinata (run) prodotta da Snow Plow è $2M$. Varianti chieste: quanto vale se la probabilità di finire nell'unsorted bucket è $\frac{1}{4}$ invece di $\frac{1}{2}$? (15/01/24) Se è $\frac{1}{8}$? (21/06/24) Se la probabilità di andare nello heap è $\frac{3}{4}$? (11/11/20)

L'algoritmo (serve descriverlo prima della prova). Snow Plow genera i run ordinati della fase 1 del MergeSort usando una memoria di M celle divisa fra un min-heap $\mathcal{H}$ e un buffer non ordinato $\mathcal{U}$. Una fase (= la produzione di un run) funziona così: all'inizio $\mathcal{H}$ contiene M item e $\mathcal{U}$ è vuoto. Ripetutamente: si estrae il minimo m da $\mathcal{H}$ e lo si scrive nel run di output; si legge il prossimo item x dallo stream: se $x \geq m$ può ancora far parte del run corrente e va in $\mathcal{H}$, altrimenti è “troppo piccolo” (uscirebbe fuori ordine) e va in $\mathcal{U}$. La memoria resta sempre piena: $|\mathcal{H}| + |\mathcal{U}| = M$. La fase termina quando $\mathcal{H}$ si svuota (equivalentemente $\mathcal{U}$ si è riempito con M item): i contenuti di $\mathcal{U}$ diventano lo heap della fase successiva. Ogni run prodotto ha lunghezza $\geq M$.

Teorema. Se ogni item letto va in $\mathcal{U}$ con probabilità $p = \frac{1}{2}$, la lunghezza attesa del run è $2M$.

Prova. Sia τ la lunghezza del run prodotto in una fase. Ogni item scritto in output provoca la lettura di un item dallo stream, quindi durante la fase vengono letti esattamente τ item. Il run è formato da: gli M item presenti nello heap all'inizio della fase, più tutti gli item letti durante la fase che sono entrati in $\mathcal{H}$ (ciascuno di essi, essendo $\geq$ del minimo corrente, verrà prima o poi estratto e scritto nello stesso run). In valore atteso gli item letti che entrano nello heap sono $(1-p)\tau$. Dunque

$$\tau = M + (1-p)\tau \quad \implies \quad p\tau = M \quad \implies \quad \tau = \frac{M}{p}$$

(passando ai valori attesi e usando la linearità). Con $p = \frac{1}{2}$: $\tau = 2M$. ■

Varianti d'esame (la formula $\tau = M/p$ le risolve tutte):

- $p = \frac{1}{4}$ (15/01/24): $\tau = 4M$;
- $p = \frac{1}{8}$ (21/06/24): $\tau = 8M$;
- probabilità di andare nello *heap* $= \frac{3}{4}$ (11/11/20): allora $p = 1 - \frac{3}{4} = \frac{1}{4}$ e $\tau = 4M$. **Attenzione:** qui p è la probabilità dell'unsorted bucket; se il testo dà quella dello heap, va complementata.

Intuizione. Più è raro che un item sia “troppo piccolo” (piccolo p), più a lungo lo heap resta rifornito e più lungo è il run. Casi limite: input già ordinato $\Rightarrow p = 0 \Rightarrow$ un unico run; input ordinato al contrario $\Rightarrow p = 1 \Rightarrow$ run di lunghezza esattamente M . Effetto sul MergeSort: run attesi $n/2M$, cioè $O\left(\frac{n}{B} \log_{M/B} \frac{n}{2M}\right)$ I/O in media.

[2.2] ★ Lower bound per l'ordinamento di stringhe

Testo dell'esercizio. Enuncia e dimostra il lower bound (comparison-based) al tempo per ordinare n stringhe di lunghezza totale N su alfabeto di taglia σ . (10/02/23, 05/06/23, orale 25/07/22)

Notazione. Per ogni stringa s del dataset S sia d_s la lunghezza del suo prefisso distinguente, cioè il più corto prefisso di s che non è prefisso di nessun'altra stringa di S ; sia $d = \sum_{s \in S} d_s$ ($d \leq N$).

Teorema. Ogni algoritmo comparison-based che ordina n stringhe distinte richiede

$$\Omega(d + n \log n)$$

confronti di singoli caratteri, nel caso peggiore.

Prova. Si dimostrano separatamente i due termini; il massimo dei due è un lower bound, e poiché $\max\{a, b\} \geq \frac{a+b}{2}$ lo è anche la somma (a meno di un fattore 2).

(i) $\Omega(n \log n)$. Argomento dell'albero di decisione: l'algoritmo deve distinguere gli $n!$ ordinamenti possibili delle stringhe; ogni confronto fra caratteri ha esito in un insieme di taglia costante (es. $<, =, >$), quindi l'albero delle computazioni ha grado costante e servono $\Omega(\log(n!)) = \Omega(n \log n)$ confronti su qualche ramo.

(ii) $\Omega(d)$. **L'algoritmo deve leggere almeno tutti i caratteri dei prefissi distinguenti.** Supponiamo per assurdo che su un input S l'algoritmo termini senza aver letto il carattere $s[i]$ con $i \leq d_s$ per qualche s . Poiché $i \leq d_s$, il prefisso $s[1, i]$ non è ancora distinguente: esiste un input alternativo, indistinguibile per l'algoritmo (coincide con S su tutti i caratteri letti), in cui il carattere in posizione i cambia l'ordine relativo di s . L'algoritmo produce lo stesso output su due input con ordinamenti diversi: contraddizione. Quindi $\Omega(d)$ letture. ■

Commento. $n \log n$ è il costo “informativo” dell'ordinamento; d è il costo di “vedere” abbastanza caratteri. **Non compare N :** i caratteri dopo il prefisso distinguente sono irrilevanti. Il Multikey QuickSort ([2.3]) raggiunge questo bound: il bound è stretto.

[2.3] ★ Multikey QuickSort: pseudocodice, complessità, ottimalità

Testo dell'esercizio. Scrivi lo pseudocodice del Multikey QuickSort su n stringhe di lunghezza totale N ; enuncia e dimostra la sua complessità temporale e mostra che è time-optimal rispetto al lower bound [2.2]. (10/02/23, 05/06/23, 07/09/23, orale 25/07/22)

Idea. È il QuickSort adattato alle stringhe: si partiziona rispetto a un solo carattere (quello in posizione i , dove i è la lunghezza del prefisso comune già accertato) in tre gruppi $<, =, >$, e solo nel gruppo $=$ si avanza al carattere successivo.

```

MultikeyQS(R, i):                                     // R insieme di stringhe, i posizione corrente
  if |R| <= 1: return R
  scegli una stringa pivot p da R uniformemente a caso; c = p[i]
  R< = { s in R : s[i] < c }
  R= = { s in R : s[i] = c }
  R> = { s in R : s[i] > c }
  return MultikeyQS(R<, i) ++ MultikeyQS(R=, i+1) ++ MultikeyQS(R>, i)

```

Invariante: tutte le stringhe di R condividono il prefisso $p[1, i-1]$, quindi confrontare il solo carattere i è corretto; la concatenazione $R_< R_ = R_>$ dà l'ordine lessicografico.

Teorema. Con pivot casuale, il tempo atteso è

$$O(d + n \log n) \text{ atteso}$$

dove d è la somma dei prefissi distinguenti.

Prova. Contiamo i confronti di carattere addebitandoli alla stringa s che li subisce. Ogni confronto colloca s in $R_<$, $R_ =$ o $R_>$:

- *Confronti di tipo “=”:* s finisce in $R_ =$ e l'indice i avanza di 1. Questo può accadere al più $d_s + O(1)$ volte: **appena i supera d_s il prefisso di s è unico**, quindi s resta da sola nel suo gruppo e la sua ricorsione termina. Totale: $O(d)$.
- *Confronti di tipo “<” o “>”:* s finisce in $R_<$ o $R_>$ e i non avanza. Questi confronti sono, in valore atteso, $O(\log n)$ per stringa: è l'analisi del QuickSort classico ristretta al carattere i -esimo. Un pivot casuale è, con probabilità $\geq \frac{1}{2}$, “centrale” (né fra i più piccoli $\frac{1}{4}$ né fra i più grandi $\frac{1}{4}$ dei caratteri i -esimi del gruppo): in tal caso $|R_<|, |R_>| \leq \frac{3}{4}|R|$. Quindi a ogni confronto di questo tipo, con probabilità costante il gruppo di s si riduce di un fattore costante: può accadere $O(\log n)$ volte. Totale: $O(n \log n)$.

Sommando: $O(d + n \log n)$ atteso. ■

Ottimalità. **Coincide col lower bound [2.2]:** il Multikey QuickSort è time-optimal (in attesa). Vantaggi ingegneristici: **non spezza le stringhe** (località), partizione ternaria in place, **non richiede di conoscere l'alfabeto**.

[2.4] ★ LSD RadixSort: complessità e correttezza

Testo dell'esercizio. Abbozza l'algoritmo LSD RadixSort per n chiavi di b bit ciascuna; enuncia e dimostra la sua complessità temporale; dimostra la correttezza dell'ordinamento prodotto (caso stringhe binarie). Variante: complessità del RadixSort ottimo su n stringhe binarie di lunghezza totale N . (16/01/23, 05/06/23, 11/07/24, 10/01/25, 23/06/25)

Algoritmo. Le n chiavi di b bit sono viste come sequenze di b/r cifre di r bit (cifre in $[0, 2^r)$). L'algoritmo fa b/r passate, **dalla cifra meno significativa alla più significativa**; ogni passata riordina l'intera sequenza rispetto alla cifra corrente con CountingSort, che è **stabile**.

```

LSD-RadixSort(S, b, r):
  for j = 1 .. b/r:                                     // j = 1 e' la cifra MENO significativa
    S = CountingSortStabile(S, cifra j)                 // O(n + 2^r) tempo
  return S

```

Complessità. Ogni passata costa $O(n + 2^r)$ (istogramma delle 2^r cifre + somme prefisse + ricollocamento). Totale:

$$O\left(\frac{b}{r}(n + 2^r)\right) \xrightarrow{r=\log_2 n} O\left(\frac{b}{\log n} \cdot n\right)$$

La scelta ottima è $r = \log_2 n$ (così $2^r = n$ non domina): per chiavi di $b = O(\log n)$ bit il costo è $O(n)$ — batte il lower bound comparison-based perché **il RadixSort non è comparison-based**.

Spazio: $O(n + 2^r)$ ausiliario. Variante (05/06/23), n stringhe binarie di lunghezza totale N (cioè $b = N/n$): tempo $O\left(\frac{nb}{\log n}\right) = O\left(\frac{N}{\log n}\right)$ (per alfabeto σ : $O\left(\frac{N \log \sigma}{\log n}\right)$).

Correttezza (induzione + stabilità). Invariante: **dopo la passata sulla cifra j , le chiavi sono ordinate rispetto alle ultime j cifre** (le j meno significative).

Base ($j = 0$): banale.

Passo. Vale l'invariante per $j - 1$; eseguiamo la passata j e consideriamo due chiavi α, β qualunque:

- **cifra j diversa:** il CountingSort le ordina secondo la cifra j , che è la più significativa fra le ultime j : ordine corretto qualunque fosse prima;
- **cifra j uguale:** il loro ordine è deciso dalle ultime $j - 1$ cifre; per ipotesi induttiva erano già nell'ordine giusto, e **la stabilità del CountingSort preserva tale ordine**.

Dopo la passata $j = b/r$: ordinate rispetto a tutte le cifre. ■

Nota: la **stabilità è essenziale** — con un ordinamento interno non stabile l'invariante si rompe sui pareggi della cifra corrente.

[2.5] ★ Bounded QuickSort: stack $O(\log n)$

Testo dell'esercizio. *Scrivi e commenta lo pseudocodice del Bounded QuickSort, cioè la variante che garantisce uno stack di taglia $O(\log n)$ ($O(\log n)$ chiamate ricorsive annidate per qualunque input).* (10/01/25, 04/02/25, 23/06/25)

Problema. Il QuickSort ricorsivo **standard**, nel caso peggiore (partizioni sbilanciate), ha **profondità di ricorsione $O(n)$** : rischio di stack overflow. La variante **bounded** elimina una delle due chiamate ricorsive con l'iterazione (tail recursion removal), **ricorrendo sempre e solo sul lato più piccolo**:

```
BoundedQuickSort(S, i, j):           // ordina S[i..j]
  while i < j:
    p = Partition(S, i, j)           // pivot (casuale o mediana di campione)
    if (p - i) < (j - p):             // lato sinistro piu' piccolo
      BoundedQuickSort(S, i, p-1)
      i = p + 1                       // il lato destro si tratta iterativamente
    else:                             // lato destro piu' piccolo (o uguale)
      BoundedQuickSort(S, p+1, j)
      j = p - 1                       // il lato sinistro si tratta iterativamente
```

Commento / prova del bound sullo stack. La chiamata ricorsiva è sempre eseguita sul lato di taglia $\leq \frac{j-i+1}{2}$. Quindi, **scendendo di un livello di ricorsione la taglia del sottoproblema almeno si dimezza**: dopo t livelli annidati è $\leq n/2^t$, perciò

$$t \leq \log_2 n \text{ livelli di stack, worst case, per qualunque input}$$

Il lato grande non consuma stack: è gestito dal ciclo **while** restringendo $[i, j]$. Il tempo resta quello del QuickSort ($O(n \log n)$ atteso con pivot casuale, $O(n^2)$ worst case): **il bound riguarda lo spazio, non il tempo**.

14 Random Sampling — risoluzioni

[3.1]–[3.2] ★ Reservoir Sampling: pseudocodice e correttezza

Testo dell'esercizio. [3.1] *Scrivi (solo) lo pseudocodice del Reservoir Sampling.* (06/03/25, 03/06/25)
[3.2] *Data una sequenza S di n item (n ignoto) e un intero m (eventualmente $m < n/2$): mostra lo pseudocodice del Reservoir Sampling e dimostrane la correttezza, cioè che ogni item è campionato con probabilità m/n .* (02/11/23, 11/07/24)

Pseudocodice.

```
ReservoirSampling(S, m):           // n = |S| ignoto a priori
  R[1..m] = primi m elementi di S
  i = m
  while (S ha un altro elemento x):
    i = i + 1
    j = intero uniforme in [1, i]
    if j <= m: R[j] = x           // x entra, R[j] esce
  return R
```

Una sola passata, tempo $O(n)$, spazio $O(m)$; in ogni istante R è un campione uniforme del prefisso letto: adatto a stream di lunghezza sconosciuta.

Teorema (correttezza). Al termine, ogni elemento di S appartiene a R con probabilità esattamente m/n .

Prova. Consideriamo l'elemento in posizione t .

Caso $t > m$. L'elemento entra in R al passo t con probabilità $\Pr[j \leq m] = \frac{m}{t}$. Per ogni passo successivo $i > t$, l'elemento in R viene espulso sse l'elemento i -esimo entra (prob. $\frac{m}{i}$) e sceglie proprio la sua cella (prob. $\frac{1}{m}$): probabilità $\frac{m}{i} \cdot \frac{1}{m} = \frac{1}{i}$; quindi sopravvive al passo i con probabilità $\frac{i-1}{i}$. Per indipendenza delle estrazioni:

$$\Pr[t \in R] = \frac{m}{t} \cdot \prod_{i=t+1}^n \frac{i-1}{i} = \frac{m}{t} \cdot \frac{t}{t+1} \cdot \frac{t+1}{t+2} \cdots \frac{n-1}{n} = \frac{m}{t} \cdot \frac{t}{n} = \boxed{\frac{m}{n}}$$

per prodotto telescopico.

Caso $t \leq m$. Entra con probabilità 1 e deve sopravvivere ai passi $i = m+1, \dots, n$: $\Pr[t \in R] = \prod_{i=m+1}^n \frac{i-1}{i} = \frac{m}{n}$.

In entrambi i casi $\frac{m}{n}$: campionamento uniforme. ■

[3.3] Sampling con n noto (selection sampling) e correttezza per $m = 1$

Testo dell'esercizio. Data una sequenza S di n item con n noto e un intero m , scrivi lo pseudocodice dell'algoritmo di sampling che estrae m item uniformemente a caso, e dimostrane la correttezza (probabilità m/n) nel caso $m = 1$. (06/11/23; simulazioni in 13/06/22, 10/02/23, 07/09/23)

Quando n è noto si decide *al volo*, per ogni elemento, se selezionarlo, con probabilità che dipende da quanti ne mancano:

```
KnownN-Sampling(S, n, m):
  s = 0                               // già' selezionati
```

```

for i = 1 .. n:
    resta = n - i + 1          // elementi ancora da esaminare (incluso i)
    serve = m - s              // elementi ancora da selezionare
    con probabilita' serve/resta: // es.: estrai p uniforme in [0,1],
        seleziona S[i]; s = s+1 // seleziona se p <= serve/resta
    if s == m: break
return i selezionati

```

Non può selezionare più di m elementi (con $s = m$ la probabilità è 0) **né meno** (quando $serve = resta$ la probabilità è 1 per tutti i rimanenti). Una passata, $O(n)$ tempo, $O(1)$ spazio. *Nota per le simulazioni d'esame:* i valori p_i dati nel testo si confrontano con la soglia corrente $serve/resta$ (si seleziona quando $p_i \leq serve/resta$).

Correttezza per $m = 1$. L'elemento t è selezionato sse nessuno dei precedenti lo è stato e la moneta del passo t ha successo (prob. $\frac{1}{n-t+1}$):

$$\Pr[t \text{ selezionato}] = \prod_{i=1}^{t-1} \left(1 - \frac{1}{n-i+1}\right) \cdot \frac{1}{n-t+1} = \prod_{i=1}^{t-1} \frac{n-i}{n-i+1} \cdot \frac{1}{n-t+1} = \frac{n-t+1}{n} \cdot \frac{1}{n-t+1} = \boxed{\frac{1}{n}}$$

di nuovo per telescopia: uniforme. ■ (Il caso generale è analogo per induzione, e il numero di selezionati è esattamente m per costruzione.)

Confronto col Reservoir. Serve conoscere n , ma niente serbatoio né sostituzioni: ogni elemento è deciso definitivamente al suo passaggio.

15 Set Intersection — risoluzioni

Setting comune: L_1 (detta anche A) di lunghezza n e L_2 (detta B) di lunghezza $m \leq n$, ordinate crescenti. Riferimenti: merge completo $O(n+m)$ (buono per $m \approx n$); m ricerche binarie $O(m \log n)$ (buono per $m \ll n$). Gli algoritmi seguenti interpolano fra i due.

[4.1] ★ Two-level intersection

Testo dell'esercizio. Date due liste ordinate L_1, L_2 di lunghezze n e m : descrivi l'algoritmo two-level (detto anche two-level scan / two-level memory / two-level storage) per calcolarne l'intersezione; enuncia e dimostra la sua complessità. (09/02/24, 21/06/24, 11/11/24, 15/07/25)

Algoritmo. Si fissa un parametro L (taglia di blocco). Si divide L_1 in $h = n/L$ blocchi consecutivi di L elementi e si costruisce la lista di primo livello $L'_1 = \{\text{primo elemento di ogni blocco}\}$, di lunghezza n/L . Fase 1: merge di L'_1 con L_2 : costo $O(\frac{n}{L} + m)$; per ogni $b \in L_2$ questo individua l'unico blocco di L_1 che può contenerlo. Fase 2: per ogni blocco selezionato, merge locale del blocco con i suoi candidati; ogni elemento di L_2 fa scandire al più un blocco: costo $O(mL + m)$ nel caso peggiore.

Teorema. Il costo è $O(\frac{n}{L} + mL + m)$; con la scelta ottima $L = \sqrt{n/m}$:

$$\boxed{T(n, m) = O(m + \sqrt{nm})}$$

Prova. La somma delle due fasi è $f(L) = \frac{n}{L} + mL$ (più $O(m)$ additivo). Minimizzo: $f'(L) = -\frac{n}{L^2} + m = 0 \iff L = \sqrt{n/m}$, dove $f(L) = 2\sqrt{nm}$ (equivalentemente, per AM-GM, $\frac{n}{L} + mL \geq 2\sqrt{nm}$ con uguaglianza in $L = \sqrt{n/m}$). ■

Commento. $\sqrt{nm}$ batte sia il merge $O(n+m)$ (quando $m \ll n$) sia le ricerche binarie $O(m \log n)$ (quando m è grande). In I/O: L_1 è scandita per blocchi contigui, quindi $O(\frac{n}{LB} + \frac{m}{B} + m)$ I/O con blocco = pagina di disco (versione “two-level memory” del 09/02/24: primi elementi in memoria, un fetch di blocco per elemento di L_2).

[4.2] Mutual partition

Testo dell'esercizio. Descrivi l'algoritmo mutual partition per l'intersezione di L_1, L_2 , con la sua complessità. (11/07/24)

Algoritmo. Si prende l'elemento mediano b_{med} della lista corta L_2 ; lo si cerca con ricerca binaria in L_1 , trovando la posizione j tale che $L_1[1, j] < b_{med} \leq L_1[j+1, n]$ (se $b_{med} \in L_1$ lo si emette). Si ricorre sulle due metà: $(L_1[1, j], L_2[1, med-1])$ e $(L_1[j+1, n], L_2[med+1, m])$. Base: lista vuota.

Complessità: $O\left(m \log \frac{2n}{m}\right)$. Al livello k della ricorsione ci sono al più 2^k sottoproblemi; le porzioni di L_1 di uno stesso livello sono disgiunte e sommano a $\leq n$; ogni sottoproblema fa una ricerca binaria $O(1 + \log n_{k,t})$ sulla propria porzione. Per concavità del logaritmo (Jensen) il costo del livello è $\sum_t (1 + \log n_{k,t}) \leq 2^k (1 + \log \frac{n}{2^k})$. Sommando sui livelli $k = 0, \dots, \log m$:

$$\sum_{k=0}^{\log m} 2^k \left(1 + \log \frac{n}{2^k}\right) = O(m) + \log n \sum_k 2^k - \sum_k k 2^k = O(m + m \log n - m \log m) = O\left(m \log \frac{2n}{m}\right),$$

usando $\sum_{k \leq K} 2^k = O(m)$ e $\sum_{k \leq K} k 2^k = (K-1)2^{K+1} + 2$ con $K = \log m$. ■

Commento. È ottimo: lower bound information-theoretic $\Omega(\log \binom{n}{m}) = \Omega(m \log \frac{n}{m})$ (bisogna distinguere gli interleaving possibili). Svantaggio: accessi non sequenziali (ricorsione + binarie), località scarsa; il doubling [4.3] ottiene lo stesso bound con accesso iterativo.

[4.3] Doubling / galloping search

Testo dell'esercizio. Descrivi il doubling algorithm (binary search con salti esponenziali, “galloping”) per l'intersezione; enuncia e dimostra la sua complessità. (15/01/24)

Algoritmo. Si scandisce $L_2 = b_1 < \dots < b_m$ mantenendo in L_1 la posizione i_{j-1} dell'ultimo attraversamento. Per il prossimo b_j : da i_{j-1} si fanno salti esponenziali $1, 2, 4, 8, \dots$ finché non si supera un elemento $\geq b_j$; detto 2^{k_j} l'ultimo salto, si fa ricerca binaria di b_j nell'intervallo di ampiezza $\Delta_j \leq 2^{k_j}$ così individuato; si emette b_j se presente e si aggiorna i_j .

Teorema. Costo totale $O\left(m \left(1 + \log \frac{n}{m}\right)\right)$.

Prova. Salti + binaria per b_j : $O(1 + \log \Delta_j)$. Poiché i salti raddoppiano, $\Delta_j \leq 2(i_j - i_{j-1})$: l'overshooting è al più il doppio del progresso reale, quindi $\sum_j \Delta_j \leq 2n$. Per concavità del logaritmo (Jensen):

$$\sum_{j=1}^m O(1 + \log \Delta_j) = O\left(m + m \log \left(\frac{1}{m} \sum_j \Delta_j\right)\right) = O\left(m + m \log \frac{2n}{m}\right). \blacksquare$$

Commento. Stesso bound ottimo del mutual partition, ma con scansione monotona di L_1 : località migliore (cache/prefetching), niente ricorsione. È l'algoritmo dei motori di ricerca sulle posting list.

[4.4] Intersezione di liste Elias–Fano con Access/GEQ

Testo dell'esercizio. *Scrivi lo pseudocodice per intersecare due liste ordinate L_1, L_2 (di n_1 e n_2 elementi) codificate con Elias–Fano, usando come primitive $\text{Access}(L, i)$ e $\text{GEQ}(L, x)$; valuta e commenta la complessità.* (14/01/22)

Primitive ($O(1)$ con le strutture di rank/select sui bit alti, [8.2]): $\text{Access}(L, i) = i$ -esimo elemento; $\text{GEQ}(L, x) =$ posizione del più piccolo elemento $\geq x$ (o $\perp$).

```
EF-Intersect(L1, L2):           // n1 = |L1|, n2 = |L2|
  i = 1
  x = Access(L1, 1)
  loop:
    j = GEQ(L2, x);   if j = NIL: return           // L2 esaurita
    y = Access(L2, j)
    if y == x:         // match
      output x
      i = i + 1;   if i > n1: return
      x = Access(L1, i)
    else:              // y > x: scavalca in L1
      i = GEQ(L1, y);   if i = NIL: return
      x = Access(L1, i)
```

Complessità. Ogni iterazione avanza strettamente almeno uno dei due puntatori (su match avanza i ; altrimenti x salta a un valore $\geq y > x$). Le iterazioni sono quindi $O(n_1 + n_2)$, e $O(\min(n_1, n_2))$ chiamando GEQ dalla lista corta verso la lunga; ogni operazione è $O(1)$: totale $O(\min(n_1, n_2))$ senza mai decomprimere le liste. Vantaggio rispetto a [4.3] su liste esplicite: spazio ($\approx \log \frac{u}{n} + 2$ bit per intero) e quindi meno I/O e cache miss; GEQ fa il ruolo dei salti esponenziali in $O(1)$.

16 Hashing, Bloom Filter, Skip List, Treap — risoluzioni

[5.1] ★ Hashing universale: definizione, esempio, prova

Testo dell'esercizio. *Definisci una classe di funzioni hash universali; fornisci un esempio e dimostrane l'universalità.* (16/01/23, 07/09/23, 02/11/23, 10/01/25)

Definizione. Una famiglia $\mathcal{H} \subseteq \{h : U \rightarrow [0, m)\}$ è universale se per ogni coppia di chiavi distinte $x \neq y$ di U :

$$\Pr_{h \in \mathcal{H}}[h(x) = h(y)] \leq \frac{1}{m}$$

dove h è estratta uniformemente da $\mathcal{H}$. Equivalentemente: il numero di funzioni della famiglia che fanno collidere x e y è $\leq |\mathcal{H}|/m$. La casualità sta nella scelta della funzione, non nelle chiavi: la garanzia vale per ogni input, anche avversario.

Esempio (prodotto scalare modulo primo). Sia m primo e si scrivano le chiavi in base m : $k = (k_0, k_1, \dots, k_{r-1})$ con $k_i \in [0, m)$. Per ogni vettore $a = (a_0, \dots, a_{r-1}) \in [0, m)^r$ si definisce

$$h_a(k) = \left(\sum_{i=0}^{r-1} a_i k_i \right) \bmod m,$$

con $\mathcal{H} = \{h_a\}$, $|\mathcal{H}| = m^r$ (scegliere h a caso = scegliere a a caso).

Prova di universalità. Siano $x \neq y$; differiscono in almeno una cifra, diciamo la d -esima: $x_d \neq y_d$. La **collisione** $h_a(x) = h_a(y)$ equivale a

$$a_d(x_d - y_d) \equiv -\sum_{i \neq d} a_i(x_i - y_i) \pmod{m}.$$

Poiché m è primo e $x_d - y_d \not\equiv 0$, $(x_d - y_d)$ è invertibile in $\mathbb{Z}_m$: fissati comunque gli altri $r - 1$ coefficienti, esiste **esattamente un** valore di a_d che soddisfa l'equazione:

$$a_d \equiv -(x_d - y_d)^{-1} \sum_{i \neq d} a_i(x_i - y_i) \pmod{m}.$$

Le funzioni che fanno collidere x, y sono dunque m^{r-1} , e $\Pr[h_a(x) = h_a(y)] = \frac{m^{r-1}}{m^r} = \frac{1}{m}$: universale. ■

[5.2] Universale + chaining: catene attese $O(1)$

Testo dell'esercizio. Definisci una classe universale e mostra che, usata nell'hashing con chaining, ottiene lunghezza attesa $O(1)$ per catena quando $m = O(n)$; da qui il calcolo del numero atteso di collisioni. (06/11/23; esercizi collegati in 10/07/20 e 11/11/20)

Definizione di classe universale: come [5.1]. Sia $T[0, m-1]$ una tabella con chaining, h estratta da una famiglia universale, n chiavi inserite. Fissata una chiave x , sia C_x il numero di chiavi $y \neq x$ che collidono con x . **Per linearità dell'attesa e universalità:**

$$\mathbb{E}[C_x] = \sum_{y \neq x} \Pr[h(y) = h(x)] \leq \frac{n-1}{m} < \alpha := \frac{n}{m}.$$

Quindi il costo atteso di una ricerca è $O(1 + \alpha)$, che è $O(1)$ non appena $m = \Omega(n)$: $m = O(n)$ **celle bastano per catene di lunghezza attesa costante.** ■

Esercizio collegato (10/07/20, 11/11/20): collisioni totali attese. Il numero di coppie collidenti è $C = \sum_{\{x,y\}} \mathbf{1}[h(x) = h(y)]$ su $\binom{n}{2}$ coppie:

$$\mathbb{E}[C] \leq \binom{n}{2} \frac{1}{m} = \frac{n(n-1)}{2m}.$$

Con i numeri d'esame ($n = 10$, range $[0, 22] \Rightarrow m = 23$): $\mathbb{E}[C] \leq \frac{10 \cdot 9}{2 \cdot 23} = \frac{45}{23} \approx 1.96$.

[5.3] ★ Cuckoo hashing: teorema dei cammini

Testo dell'esercizio. Enuncia e dimostra il teorema sul cuckoo graph: per ogni coppia di posizioni i, j della tabella cuckoo, la probabilità che il cammino minimo $i \rightarrow j$ abbia lunghezza L è al più c^{-L}/r , dove r è il numero di posizioni della tabella, n le chiavi memorizzate e $r \geq 2cn$. Commenta l'impatto sul load factor. (10/01/20, 10/02/23, 03/06/25)

Contesto. Nel cuckoo hashing ogni chiave k ha **due posizioni possibili** $h_1(k), h_2(k)$ in una tabella di r celle; l'inserimento sfrutta a catena. Il *cuckoo graph* ha come **nodi le r celle** e, per ogni chiave, un **arco fra le sue due posizioni**. Le prestazioni dipendono dall'**assenza di cammini lunghi e cicli**.

Teorema. Se le funzioni hash sono (idealmente) casuali e $r \geq 2cn$ per una costante $c > 1$, allora per ogni coppia di celle i, j e ogni $L \geq 1$:

$$\Pr[\exists \text{ cammino minimo } i \rightarrow j \text{ di lunghezza } L] \leq \frac{c^{-L}}{r}$$

Prova (induzione su L).

Base $L = 1$. Esiste un arco (i, j) sse qualche chiave k ha $\{h_1(k), h_2(k)\} = \{i, j\}$. Per una singola chiave: $\Pr = \frac{2}{r^2}$ (due orientamenti). Per union bound sulle n chiavi, **usando l'ipotesi $r \geq 2cn$, cioè $n \leq r/(2c)$** :

$$\Pr[\exists \text{ arco } i \rightarrow j] \leq \frac{2n}{r^2} \leq \frac{2}{r^2} \cdot \frac{r}{2c} = \frac{c^{-1}}{r}.$$

Passo. Un cammino minimo $i \rightarrow j$ di lunghezza L è composto da un cammino minimo di lunghezza $L - 1$ da i a una cella intermedia ℓ , seguito da un arco (ℓ, j) . Per union bound su ℓ , con ipotesi induttiva e indipendenza dell'ultimo arco:

$$\Pr[\text{cammino } i \rightarrow j \text{ di lung. } L] \leq \sum_{\ell=1}^r \frac{c^{-(L-1)}}{r} \cdot \frac{c^{-1}}{r} = r \cdot \frac{c^{-(L-1)}}{r} \cdot \frac{1}{cr} = \frac{c^{-L}}{r}. \blacksquare$$

Conseguenze e load factor. Probabilità che i e j siano connessi (lunghezza qualunque): $\sum_{L \geq 1} \frac{c^{-L}}{r} = \frac{1}{r(c-1)}$. Da qui: (i) **insert atteso $O(1)$** (cammini attesi di lunghezza costante) **e ricerca/delete $O(1)$ worst case** (si guardano 2 celle); (ii) la **probabilità di un ciclo** (fallimento $\Rightarrow$ rehash) **si maggiora con $\frac{1}{c-1}$** : con $c = 3$ è $\leq \frac{1}{2}$, quindi $O(1)$ rehash attesi, ammortizzato $O(1)$. **Prezzo:** $r \geq 2cn = 6n$ celle, cioè **load factor $\leq 1/6 \approx 17\%$** : il cuckoo classico garantisce i bound solo a tabelle molto scariche (varianti con più hash o celle multi-slot arrivano oltre il 90%).

[5.4]–[5.5] ★ Bloom Filter: definizione, bit a 0, falso positivo, ottimo

Testo dell'esercizio. [5.4] Calcola la probabilità che una posizione del bit-array del Bloom Filter (taglia m , n chiavi inserite) sia 0, e deriva/dimostra la probabilità d'errore complessiva. Variante: dimostra che tale probabilità è $\frac{1}{2}$ quando il numero di funzioni hash è quello ottimo. (11/11/20, 27/05/24, 11/11/24, 15/07/25)
[5.5] Definisci un Bloom Filter su un dizionario di n chiavi e le operazioni supportate; enuncia e dimostra il risultato sul falso positivo. (24/07/23)

Definizione e operazioni ([5.5]). Un Bloom Filter per un dizionario D di n chiavi è un **array binario** $B[0, m-1]$, inizialmente tutto 0, più k funzioni hash indipendenti $h_1, \dots, h_k : U \rightarrow [0, m)$. **Insert(x)**: pone $B[h_i(x)] = 1$ per $i = 1..k$; costo $O(k)$. **Query(y)**: risponde “presente” sse $B[h_i(y)] = 1$ per tutti gli i ; costo $O(k)$. **Non supporta la cancellazione** (creerebbe falsi negativi; la variante counting/spectral usa contatori). Proprietà fondamentale: **niente falsi negativi, possibili falsi positivi**.

Probabilità che un bit sia 0. Gli **inserimenti** accendono **kn posizioni casuali indipendenti**; una posizione fissata resta 0 se tutte le kn estrazioni la evitano:

$$p_0 = \left(1 - \frac{1}{m}\right)^{kn} \approx e^{-kn/m}$$

Probabilità di falso positivo. $y \notin D$ è dichiarata presente sse le sue k posizioni sono tutte a 1 (assumendo indipendenza, approssimazione standard):

$$\epsilon = (1 - p_0)^k \approx (1 - e^{-kn/m})^k$$

Numero ottimo di hash e $p_0 = \frac{1}{2}$ ([5.4]). Minimizziamo $\varepsilon(k) = (1 - e^{-kn/m})^k$. Posto $t = e^{-kn/m}$, cioè $k = -\frac{m}{n} \ln t$:

$$\ln \varepsilon = k \ln(1 - t) = -\frac{m}{n} \ln t \ln(1 - t).$$

La funzione $g(t) = \ln t \ln(1 - t)$ è **simmetrica rispetto a $t \mapsto 1 - t$** e unimodale: il punto **ottimo** è $t = \frac{1}{2}$ (formalmente $g'(t) = \frac{\ln(1-t)}{t} - \frac{\ln t}{1-t} = 0$ in $t = \frac{1}{2}$). Quindi

$$e^{-k^*n/m} = \frac{1}{2} \implies \boxed{k^* = \frac{m}{n} \ln 2, \quad p_0 = \frac{1}{2}}$$

con k ottimo metà dei bit è a 0 e metà a 1 (massima “entropia” del filtro). **Errore** all’ottimo:

$$\boxed{\varepsilon^* = \left(\frac{1}{2}\right)^{k^*} = 2^{-\frac{m}{n} \ln 2} \approx (0.6185)^{m/n}}$$

esponenzialmente piccolo nel numero di bit per chiave m/n (es. $m/n = 10 \Rightarrow \varepsilon \approx 0.8\%$ con $k^* \approx 7$).

[5.6] Intersezione di due insiemi su due server con un Bloom Filter

Testo dell’esercizio. Dati due insiemi A, B memorizzati su due server, mostra come calcolarne l’intersezione usando un Bloom Filter e un solo round di comunicazione, ammettendo errori. (05/06/23)

Protocollo. Il server 1 costruisce un Bloom Filter BF_A di A (taglia $m = \Theta(|A|)$ bit, k ottimo) e **lo invia al server 2** (un solo messaggio). Il server 2 esegue $Query_{BF_A}(b)$ per ogni $b \in B$ e restituisce $X = \{b \in B : BF_A(b) = \text{presente}\}$.

Correttezza/errore. **Niente falsi negativi $\Rightarrow A \cap B \subseteq X$** : nessun elemento dell’intersezione è perso. Ogni $b \in B \setminus A$ finisce in X con probabilità $\varepsilon \approx (0.6185)^{m/|A|}$: attesi $\varepsilon |B \setminus A|$ falsi positivi, **controllabili aumentando m** . **Comunicazione:** $m = O(|A|)$ **bit** invece di $\Theta(|A| \log u)$ bit per spedire A esplicitamente. (Se serve l’intersezione esatta, un secondo round di verifica elimina gli errori — ma la domanda chiede la versione a un round.)

[5.7] Skip List: altezza media $O(\log n)$

Testo dell’esercizio. Dimostra che l’altezza (profondità) media di una Skip List con n nodi è $O(\log n)$. (11/11/24, 04/02/25)

Setup. Ogni elemento viene promosso di livello con probabilità $\frac{1}{2}$ indipendentemente: $\Pr[\text{liv}(x) \geq \ell] = 2^{-(\ell-1)}$ (geometrica). **L’altezza è $H = \max_x \text{liv}(x)$** .

Prova. Per union bound, per ogni $\ell \geq 1$:

$$\Pr[H \geq \ell] \leq \sum_x \Pr[\text{liv}(x) \geq \ell] = n 2^{-(\ell-1)} \quad (\text{e ovviamente } \leq 1).$$

Con l’identità $\mathbb{E}[H] = \sum_{\ell \geq 1} \Pr[H \geq \ell]$, spezzando la somma in $\ell \leq 1 + \log_2 n$ e $\ell > 1 + \log_2 n$:

$$\mathbb{E}[H] \leq \underbrace{\sum_{\ell=1}^{1+\log_2 n} 1}_{\leq 1+\log_2 n} + \sum_{j \geq 1} n 2^{-(\log_2 n + j)} = 1 + \log_2 n + \sum_{j \geq 1} 2^{-j} = \boxed{\log_2 n + 2} = O(\log n). \blacksquare$$

Concentrazione: $\Pr[H \geq c \log_2 n] \leq 2n^{1-c}$, cioè **altezza $O(\log n)$ con alta probabilità**.

Complemento utile (ricerca $O(\log n)$ attesa). Analizzando il cammino di ricerca *a ritroso*: da ogni nodo si sale di livello con probabilità $\frac{1}{2}$, altrimenti ci si sposta orizzontalmente; per salire gli $O(\log n)$ livelli servono in attesa 2 passi per livello: $O(\log n)$ passi totali.

[5.8] ★ Treap: definizione, operazioni, split

Testo dell'esercizio. Definisci la struttura Treap su n coppie $\langle \text{chiave}, \text{priorità} \rangle$ con priorità minima nella radice; descrivi le operazioni supportate e come si implementa lo split su una chiave K non presente. (10/02/23, 23/06/25, orale 25/07/22)

Definizione. Un Treap su n coppie $\langle k, \pi \rangle$ è un albero binario che è contemporaneamente: (i) un BST rispetto alle chiavi k (visita in order = ordine crescente delle chiavi); (ii) uno heap rispetto alle priorità π — qui un MIN-heap: la radice ha la priorità minima. Se le priorità sono distinte, il Treap è unico: la radice è forzata (priorità minima), le chiavi minori vanno a sinistra, le maggiori a destra, ricorsivamente. Operazioni: search ($O(h)$ come BST), insert e delete tramite rotazioni che ripristinano lo heap ($O(h)$), split e merge ($O(h)$); $h = O(\log n)$ attesa con priorità casuali ([5.9]).

Split su chiave K non presente. Obiettivo: dividere T in $T_{<K}$ e $T_{>K}$.

1. Si inserisce il nodo fittizio $\langle K, -\infty \rangle$ con l'insert standard: discesa BST fino a una foglia, poi — avendo priorità minima di tutti — il nodo risale con rotazioni fino a diventare la radice.
2. Per la proprietà BST, il sottoalbero sinistro della nuova radice contiene esattamente le chiavi $< K$ e il destro le chiavi $> K$ (le rotazioni preservano BST e heap nei sottoalberi).
3. Si rimuove il nodo fittizio: i suoi due figli sono $T_{<K}$ e $T_{>K}$.

Costo: un insert = $O(\log n)$ atteso. (Il merge è l'operazione inversa: radice fittizia sopra i due treap, che poi si fa scendere ed eliminare.)

[5.9] Treap: profondità media $O(\log n)$

Testo dell'esercizio. Enuncia e dimostra che la profondità media di un Treap con priorità scelte uniformemente a caso è logaritmica nel numero di chiavi. (07/02/20, orale 25/07/22)

Teorema. Se le priorità sono uniformi e indipendenti, per ogni nodo $\mathbb{E}[\text{depth}(u_j)] \leq 2 \ln n = O(\log n)$.

Lemma (struttura degli antenati). Siano $u_1, \dots, u_n$ i nodi in ordine di chiave. Allora u_i è antenato di $u_j \iff u_i$ ha la priorità minima nell'intervallo di chiavi $S_{ij} = \{u_{\min(i,j)}, \dots, u_{\max(i,j)}\}$.

Prova del lemma. Sia z il nodo di priorità minima in S_{ij} . Nel (unico) Treap, il più piccolo sottoalbero che contiene tutto S_{ij} ha come radice un nodo di S_{ij} (le chiavi di S_{ij} sono contigue, e la radice che le separa ha chiave interna all'intervallo); per la proprietà di heap tale radice è z , che quindi è antenato di tutti i nodi di S_{ij} . Se $z = u_i$: u_i antenato di u_j . Se $z \neq u_i$: o $z = u_j$ (e allora è u_j l'antenato), oppure u_i e u_j stanno in sottoalberi diversi di z (chiavi da parti opposte della chiave di z) e u_i non è antenato di u_j . In tutti i casi: antenato $\iff$ minimo. $\square$

Prova del teorema. Con priorità uniformi, ogni nodo di S_{ij} ($|i - j| + 1$ elementi) è il minimo con la stessa probabilità:

$$\Pr[u_i \text{ antenato di } u_j] = \frac{1}{|i - j| + 1}.$$

La profondità di u_j è il numero dei suoi antenati; per linearità dell'attesa:

$$\mathbb{E}[\text{depth}(u_j)] = \sum_{i \neq j} \frac{1}{|i - j| + 1} = \sum_{t=2}^j \frac{1}{t} + \sum_{t=2}^{n-j+1} \frac{1}{t} \leq 2(H_n - 1) \leq \boxed{2 \ln n} = O(\log n). \blacksquare$$

Commento. Il Treap con priorità casuali è un random BST, qualunque sia l'ordine di inserimento delle chiavi: è la sua ragion d'essere.

17 Prefix Search — risoluzioni

[6.1] Ricerca lessicografica in un Patricia Trie

Testo dell'esercizio. Dato un Patricia Trie su un dizionario S di n stringhe, descrivi come cercare la posizione lessicografica di $P[1,p]$ in S ; enuncia la complessità in tempo e in I/O. (24/07/23)

Struttura (richiamo). Il Patricia Trie è il compacted trie in cui ogni arco memorizza soltanto il primo carattere dell'etichetta, e ogni nodo la lunghezza (skip) del prefisso raggiunto; le stringhe complete risiedono solo nelle foglie (tipicamente su disco).

Ricerca (blind search), tre passi:

1. **Discesa cieca.** Dalla radice, a ogni nodo con skip ℓ si usa $P[\ell+1]$ per scegliere l'arco (confrontando solo il carattere di diramazione, **senza verificare i caratteri saltati**). Si termina a una foglia candidata f (in caso di mismatch di diramazione si prosegue comunque su un rappresentante del sottoalbero).
2. **Verifica.** Si carica l'unica stringa s_f della foglia e si calcola $\ell^* = \text{lcp}(P, s_f)$. Proprietà chiave che rende corretta la blind search: s_f massimizza l'lcp con P fra tutte le stringhe di S (le stringhe con lcp maggiore starebbero sullo stesso cammino di discesa).
3. **Riposizionamento.** Si risale al punto di profondità ℓ^* del cammino; confrontando $P[\ell^* + 1]$ con i caratteri di diramazione si determina la posizione di P fra i sottoalberi, e contando le foglie a sinistra la sua posizione lessicografica in S .

Complessità.

$O(p)$ tempo,	$O(1)$ I/O
---------------	------------

(più precisamente $O(p/B)$ I/O per leggere la stringa candidata). **Una sola stringa toccata per qualunque ricerca:** è il punto di forza del Patricia.

[6.2]–[6.3] Compacted Trie vs Patricia Trie, e l'indice two-level

Testo dell'esercizio. [6.2] Descrivi la differenza fra Compacted Trie e Patricia Trie in termini di spazio e tempo di query, in funzione di n , della lunghezza totale N e della lunghezza del pattern P . (06/03/25)

[6.3] Qual è la differenza principale fra Compacted Trie e Patricia Trie, e come è stata sfruttata per costruire un indice two-level efficiente per un insieme di stringhe? (03/06/25)

Compacted trie. Archi etichettati con sottostringhe intere (coppie (puntatore, lunghezza) nelle stringhe di S). Nodi $O(n)$, ma per navigare servono le stringhe: spazio complessivo $\Theta(N)$ caratteri + $O(n)$ puntatori. Query: confronta tutti i caratteri di P durante la discesa: tempo $O(p)$, ma se le stringhe sono su disco ogni arco può costare accessi (più di $O(1)$ I/O).

Patricia trie. Un carattere + skip per arco: spazio $O(n)$ parole, indipendente da N ; query $O(p)$ con un solo accesso a una stringa per la verifica finale: $O(1)$ I/O ([6.1]). Prezzo: i caratteri saltati non sono controllati in discesa (serve la verifica).

Differenza principale ([6.3]). Il compacted trie richiede le etichette complete (spazio $\propto N$); il Patricia riduce ogni etichetta a 1 carattere + 1 intero: spazio indipendente dalla lunghezza delle stringhe, al costo di una verifica posticipata.

Sfruttamento nel two-level index. Dizionario ordinato su disco a blocchi; in memoria un Patricia trie su una stringa campione per blocco (es. la prima):

- lo spazio in memoria è $O(n/B_{\text{blk}})$ parole e **non dipende dalla lunghezza delle stringhe campione** — la proprietà sfruttata è esattamente questa;
- una query fa blind search in $O(p)$ tempo e $O(1)$ I/O, individuando **l'unico blocco** che può contenere P ;
- si legge quel blocco ($O(1)$ I/O + $O(\text{occ}/B)$ per output lunghi) e lo si scandisce; i blocchi sono compressi con *front coding*, decodificabile dal capo-blocco memorizzato per intero.

Risultato: **prefix search in $O(p)$ tempo e $O(1 + \text{occ}/B)$ I/O con memoria interna minima.**

[6.4] Storage di un dizionario di stringhe: tre soluzioni a confronto

Testo dell'esercizio. Dato un dizionario D di n stringhe di lunghezza variabile e lunghezza totale N , discuti almeno 3 soluzioni di memorizzazione, commentando spazio e costo tempo/I/O di $\text{Access}(i)$. (27/05/24)

(1) **Concatenazione + array di offset.** Stringhe concatenate in un array di N caratteri; array $\text{off}[1, n]$ ($n \lceil \log_2 N \rceil$ bit). $\text{Access}(i)$: lettura diretta di $[\text{off}[i], \text{off}[i+1]]$: **tempo $O(|s_i|)$, I/O $O(1 + |s_i|/B)$** — ottimo. Spazio: N caratteri + $n \log N$ bit (gli offset possono dominare con stringhe corte). Nessuna compressione.

(2) **Concatenazione con lunghezze in linea (senza puntatori).** Ogni stringa preceduta dalla propria lunghezza (es. in γ -code): spazio $\approx N + O(n \log \frac{N}{n})$ bit. $\text{Access}(i)$ richiederebbe la scansione dall'inizio: **si corregge campionando un offset esplicito ogni k stringhe** (spazio extra $\frac{n}{k} \log N$ bit): Access = salto al campione + scansione di $\leq k$ stringhe: tempo $O(k \ell_{\max})$, I/O $O(1 + k \ell_{\max}/B)$. **Trade-off spazio/tempo regolato da k .**

(3) **Front coding a blocchi (FC_B).** Dizionario ordinato: stringhe consecutive condividono prefissi lunghi. **Blocchi di k stringhe: la prima per intero** (copia di ripartenza), **le successive come coppie $(\ell_j, \hat{s}_j)$** (lcp con la precedente, suffisso residuo). Spazio: $\text{FC}_B(D) \ll N$ su dizionari reali ($\approx -70\%$ sugli URL) + $\frac{n}{k} \log N$ bit di puntatori. $\text{Access}(i)$: salto al blocco $\lceil i/k \rceil$ ($O(1)$ I/O) e decodifica in scansione: tempo $O(k \ell_{\max})$, I/O tipicamente $O(1)$. **È la soluzione usata nei blocchi dell'indice two-level di [6.3].**

Confronto: (1) accesso ottimo, spazio massimo; (2) spazio minore, accesso più lento (regolabile); (3) **compressione vera con accesso quasi-ottimo: il compromesso standard.** (Quarta opzione citabile: compacted trie/Patricia + stringhe su disco, se servono anche ricerche per prefisso.)

[6.5] Ordinare stringhe con un compacted trie a fan-out hash

Testo dell'esercizio. Descrivi come usare un compacted trie (con fan-out dei nodi in hash table) per **ordinare n stringhe di lunghezza totale L** ; commenta tempo e spazio. (10/07/20)

Costruzione. Si inseriscono **le n stringhe una alla volta nel compacted trie, dove il fan-out di ogni nodo è una hash table** (carattere di diramazione $\rightarrow$ figlio). L'inserimento di s scende carattere per carattere: ogni passo costa $O(1)$ atteso (lookup hash); in caso di mismatch a metà arco si spezza l'arco con $O(1)$ nodi nuovi. Costruzione: **$O(L)$ atteso; spazio $O(n)$ nodi** + etichette come puntatori nelle stringhe originali.

Ordinamento. L'ordine lessicografico è la **DFS del trie con i figli visitati in ordine alfabetico.** Ma **le hash table non mantengono l'ordine**: prima della DFS si ordinano le liste dei figli di ciascun nodo: $\sum_v O(d_v \log d_v) \leq O(n \log \sigma)$ (poiché $\sum_v d_v \leq 2n$ e $d_v \leq \sigma$). DFS finale: $O(n)$.

Bilancio.

$O(L + n \log \sigma)$ tempo atteso,	$O(n)$ spazio extra
--------------------------------------	---------------------

Confronto col lower bound [2.2] $\Omega(d + n \log n)$: il trie evita di riconfrontare i prefissi comuni e sposta il termine di ordinamento a $n \log \sigma$; il prezzo è il bound solo *atteso* (hashing) e la località peggiore rispetto al Multikey QuickSort.

18 Substring Search — risoluzioni

[7.1] Definizioni: Suffix Array e array LCP

Testo dell'esercizio. Definisci formalmente il suffix array SA di un testo $T[1, n]$ e il corrispondente array LCP . (16/01/23, 02/11/23)

Suffix Array. Dato $T[1, n]$ (terminato da \$, minimo lessicografico), $SA[1, n]$ è la permutazione di $\{1, \dots, n\}$ che ordina lessicograficamente i suffissi:

$$T[SA[1], n] < T[SA[2], n] < \dots < T[SA[n], n],$$

cioè $SA[i]$ è la posizione iniziale dell' i -esimo suffisso in ordine lessicografico. Spazio: n interi = $\Theta(n \log n)$ bit (oltre a T).

Array LCP. $LCP[1, n - 1]$ dove

$$LCP[i] = |\text{lcp}(T[SA[i], n], T[SA[i + 1], n])|$$

è la lunghezza del più lungo prefisso comune fra i suffissi lessicograficamente consecutivi. Proprietà utile: l'lcps fra i suffissi in posizioni $i < j$ è $\min_{i \leq t < j} LCP[t]$ (range minimum).

[7.2], [7.4], [7.5] ★ Ricerca e COUNT con il Suffix Array

Testo dell'esercizio. [7.2] Definisci SA e LCP ; descrivi come cercare $P[1, p]$ in SA ; enuncia e dimostra la complessità della ricerca. (07/09/23)

[7.4] Mostra come CONTARE in $T[1, n]$ tutte le occorrenze di $P[1, p]$ con un Suffix Array in memoria; enuncia e dimostra la complessità; qual è il costo I/O del RETRIEVAL delle posizioni se SA è su disco? (10/02/21, 15/01/24)

[7.5] Descrivi come cercare $P[1, p]$ in un SA costruito su $T[1, n]$ e commenta la complessità in funzione di p e n . (15/07/25)

Osservazione chiave. Le occorrenze di P sono i suffissi che hanno P come prefisso; nell'ordine lessicografico questi suffissi sono **consecutivi**: formano un intervallo $SA[i^*, j^*]$.

Algoritmo. Due ricerche binarie su SA : una per il bordo sinistro i^* , una per il bordo destro j^* . Ogni passo confronta P col suffisso corrente: $\leq p$ confronti di caratteri.

Complessità (con prova).

$$O(p \log n) \text{ (ricerca ingenua), } O(p + \log n) \text{ con Llcps/Rlcps}$$

Prova del primo bound: la ricerca binaria fa $\lceil \log_2 n \rceil$ iterazioni; ogni iterazione è un confronto lessicografico che termina al primo mismatch, dopo al più p caratteri. *Raffinamento:* mantenendo per l'intervallo corrente $[L, R]$ i valori $\ell = \text{lcp}(P, \text{suff}_{SA[L]})$, $r = \text{lcp}(P, \text{suff}_{SA[R]})$ e usando gli array precalcolati $Llcps/Rlcps$, a ogni passo **o si dimezza l'intervallo senza leggere caratteri, o ogni carattere letto di P fa crescere $\max(\ell, r)$** ($\leq p$ volte in tutto): totale $O(p + \log n)$. ■

COUNT ([7.4]). Trovato l'intervallo: $\text{occ} = j^* - i^* + 1$, senza lavoro aggiuntivo. Costo: quello della ricerca.

RETRIEVAL con SA su disco ([7.4]). Le posizioni sono $SA[i^*..j^*]$: **celle contigue su disco**, lette con scansione sequenziale in

$$O\left(\frac{occ}{B}\right) \text{ I/O}$$

(più $O(\frac{p}{B} \log n)$ I/O per le binarie se anche T è su disco). **La contiguità rende il retrieval I/O-ottimale, output-sensitive.**

[7.3] Costruzione del Suffix Array via qsort

Testo dell'esercizio. Definisci SA ; scrivi lo pseudocodice per costruirlo via qsort; enuncia e commenta la complessità worst-case (tempo e I/O) di questa costruzione, e il caso in cui il massimo LCP fra i suffissi è limitato da una costante c . (07/09/21, 24/07/23)

Pseudocodice.

```
BuildSA-qsort(T[1,n]):
    SA = [1, 2, ..., n]           // indici dei suffissi
    qsort(SA, cmp)                // comparatore:
        cmp(i, j) = confronto lessicografico // strncmp(T+i, T+j)
                    di T[i,n] con T[j,n]      // carattere per carattere
    return SA
```

Tempo (worst case). qsort fa $O(n \log n)$ confronti; ogni confronto costa fino a $\Theta(n)$ caratteri (suffissi con lcp lungo, caso estremo $T = a^n$):

$$O(n^2 \log n) \text{ tempo worst case}$$

I/O e perché è inefficiente. Due ragioni strutturali: (a) ogni **confronto** scandisce fino a $\Theta(n)$ caratteri **a partire da due posizioni arbitrarie del testo**: $O(n/B)$ I/O a confronto, $O(\frac{n}{B} \log n)$ totali; (b) qsort permuta *puntatori*: **gli accessi ai suffissi sono sparsi e distruggono la località** (niente prefetching). È l'esempio da manuale di algoritmo RAM-accettabile ma **I/O-pessimo**; le costruzioni serie (skew/DC3, SA-IS) ottengono $O(n)$ o $O(sort(n))$.

Caso LCP massimo $\leq c$. Ogni confronto termina entro $c + 1$ caratteri, cioè $O(1)$:

$$O(n \log n) \text{ tempo}$$

Il degrado dipende dalla ripetitività del testo, misurata dagli LCP.

[7.6] Suffix tree: la più lunga sottostringa che occorre $\geq L$ volte

Testo dell'esercizio. Dato il suffix tree di $T[1,n]$, come si calcola la sottostringa più lunga che occorre almeno L volte? Complessità in funzione di n . (12/06/20)

Fatti usati. (i) ogni sottostringa di T è un **cammino dalla radice** (anche a metà arco); (ii) **il numero di occorrenze di una sottostringa = numero di foglie nel sottoalbero** del punto che la rappresenta; (iii) i **nodi interni** sono $\leq n - 1$.

Algoritmo.

1. Costruisci il **suffix tree** (dato, oppure $O(n)$ con McCreight/Ukkonen).

2. Con una **DFS** annota ogni **nodo interno v con $leaves(v)$** (somma sui figli, foglie = 1): $O(n)$.
3. Fra i nodi interni con $leaves(v) \geq L$, prendi quello di **massima string depth**: la stringa radice $\rightarrow v^*$ è la risposta.

Correttezza. A metà arco il sottoalbero (quindi il numero di occorrenze) è lo stesso del nodo sottostante: la risposta ottima può essere **estesa fino al prossimo nodo interno senza perdere occorrenze**, quindi **termina esattamente in un nodo interno**, che l'algoritmo trova. **Complessità:** $O(n)$, indipendente da L .

[7.7] Array LCP in tempo lineare (algoritmo di Kasai)

Testo dell'esercizio. *Scrivi lo pseudocodice dell'algoritmo lineare (Kasai et al.) che calcola l'array LCP dato T e SA , e dimostra che è $O(n)$.* (orale 25/07/22, domanda "da lode"; simulazioni in 11/11/24, 06/03/25, 15/07/25)

Serve l'inverso ISA ($ISA[SA[i]] = i$), calcolabile in $O(n)$.

```
Kasai(T, SA):
  for i = 1..n: ISA[SA[i]] = i          // inverso di SA
  h = 0
  for i = 1..n:                          // i = posizione NEL TESTO
    if ISA[i] > 1:
      j = SA[ISA[i] - 1]                // suffisso che precede T[i,n] in ordine less.
      while i+h <= n and j+h <= n and T[i+h] == T[j+h]:
        h = h + 1                       // estende il prefisso comune
      LCP[ISA[i] - 1] = h
      if h > 0: h = h - 1                // proprieta' chiave (vedi prova)
    else:
      h = 0                             // T[i,n] e' il minimo lessicografico
  return LCP
```

Idea e correttezza. Si processano i suffissi **in ordine di posizione nel testo**, non lessicografico, riuscendo il lavoro: se $T[i, n]$ ha $lcp = h$ col suo predecessore lessicografico $T[j, n]$, togliendo il primo carattere si ottengono $T[i+1, n]$ e $T[j+1, n]$ con prefisso comune $h-1$, e $T[j+1, n]$ è ancora minore di $T[i+1, n]$. Il predecessore *immediato* di $T[i+1, n]$ è, fra i minori, quello con lcp massimo: quindi **il suo lcp è $\geq h-1$** e il while del passo $i+1$ può ripartire da $h-1$ senza ricontrollare nulla.

Complessità $O(n)$ (prova). Il costo dominante è il **numero totale di incrementi di h** . La variabile h : parte da 0, **non supera mai n , e viene decrementata al più n volte** (una per iterazione del for, mai sotto 0). Quindi gli **incrementi sono al più (decrementi) + (massimo) $\leq 2n$** :

$$\leq 2n \text{ confronti di caratteri in totale} \Rightarrow O(n)$$

■ (È anche la risposta alla richiesta "mostra il numero di caratteri effettivamente confrontati" delle simulazioni d'esame.)

19 Integer Coding — risoluzioni

Convenzione sui bit alti (importante per gli esercizi). Nei compiti del docente la sequenza H è costruita **per bucket**: per ogni valore $v = 0, 1, \dots, u/2^\ell - 1$ della parte alta si scrivono **tanti 1 quanti sono gli elementi con parte alta = v , seguiti da uno 0 separatore** (es. $H = 110\ 110\ 100\ 10\dots$

il primo bucket ha 2 elementi, il secondo 2, il terzo 1, il quarto 0, ...). Quindi **gli 1 rappresentano gli elementi e gli 0 i separatori**. (La dispensa usa la convenzione speculare con 0 e 1 scambiati: le formule si trasformano scambiando $\text{Select}_1 \leftrightarrow \text{Select}_0$.)

[8.1] ★ Elias–Fano: costruzione e spazio (con prova)

Testo dell'esercizio. Data una sequenza monotona di n interi non negativi $< u$, enuncia e dimostra lo spazio (in bit) della codifica di Elias–Fano. (10/01/20, 27/05/24, 04/02/25)

Costruzione. Sequenza monotona $s_1 \leq \dots \leq s_n$ di interi in $[0, u)$. Si pone

$$\ell = \left\lfloor \log_2 \frac{u}{n} \right\rfloor$$

Ogni s_i è spezzato in: *parte bassa* = gli ℓ bit meno significativi, scritti in chiaro nell'array L ($|L| = n\ell$ bit); *parte alta* $h_i = \lfloor s_i/2^\ell \rfloor \in [0, u/2^\ell)$, rappresentata in H con la codifica unaria a bucket.

Teorema (spazio).

$$|L| + |H| = n\ell + \left(n + \frac{u}{2^\ell}\right) < n \left\lfloor \log_2 \frac{u}{n} \right\rfloor + 3n \text{ bit}$$

cioè meno di $\log_2 \frac{u}{n} + 3$ bit per elemento.

Prova. $|L| = n\ell$: esattamente ℓ bit per elemento. Per H : **un 1 per ciascuno degli n elementi e uno 0 per ciascuno dei $u/2^\ell$ bucket**: $|H| = n + u/2^\ell$. Resta da maggiorare $u/2^\ell$: dalla definizione di parte intera, $\ell > \log_2 \frac{u}{n} - 1$, quindi

$$2^\ell > \frac{u}{2n} \implies \frac{u}{2^\ell} < 2n.$$

Dunque $|H| < 3n$ e il totale è $< n\ell + 3n$. ■

Commento. Il minimo teorico è $\log_2 \binom{u}{n} \approx n \log_2 \frac{u}{n} + 1.44n$ bit: **Elias–Fano è a $\approx 2n$ bit dall'ottimo, qualunque sia la distribuzione**, e in cambio supporta Access e NextGEQ efficienti ([8.2]) — ciò che l'interpolativo, più compatto su sequenze clusterizzate, non offre. **Negli esercizi numerici usare il conteggio esatto $n\ell + n + u/2^\ell$.**

[8.2] Access(i) e NextGEQ(x) su Elias–Fano

Testo dell'esercizio. Scrivi l'algoritmo per eseguire Access(i) su una sequenza codificata con Elias–Fano; cenno a NextGEQ(x). (11/07/24; simulazioni in molti appelli)

Access(i) (restituisce s_i , con i da 1):

- pos = Select₁(H, i)**: posizione dell' i -esimo 1 in H ($O(1)$ con la struttura di select, [10.1]);
- parte alta: $h_i = \text{pos} - i$** — prima dell' i -esimo 1 ci sono $i - 1$ uni e $\text{pos} - i$ zeri, e il numero di separatori che precedono l'elemento è l'indice del suo bucket;
- parte bassa: $L[i]$** , cioè i bit $[(i - 1)\ell + 1, i\ell]$ di L ;
- output:

$$s_i = (\text{Select}_1(H, i) - i) \cdot 2^\ell + L[i]$$

Costo: $O(1)$ (una select + una lettura di ℓ bit consecutivi).

NextGEQ(x): sia $b = \lfloor x/2^\ell \rfloor$ il bucket di x . Con $p = \text{Select}_0(H, b)$ si trova la fine del bucket $b - 1$: gli elementi del bucket b iniziano dall'indice $j_0 = p - b + 1$. Si scandiscono le parti basse del bucket b confrontandole con $x \bmod 2^\ell$; il primo elemento $\geq x$ è la risposta; se il bucket si esaurisce, è il primo elemento del bucket non vuoto successivo. Costo: $O(1)$ select + scansione del bucket (ammortizzata $O(1)$ nell'uso tipico, [4.4]).

[8.3] Ricavare n (e ℓ) dagli array L, H

Testo dell'esercizio. Dati gli array L e H di una codifica Elias–Fano, mostra come ricavare il valore di n (numero di interi codificati), motivando la risposta. (orale 25/07/22; è il “hint” ricorrente degli esercizi: 11/11/20, 12/12/23, 09/12/24, 06/03/25, 23/06/25)

Risposta.

$$n = \#_1(H) \text{ (numero di bit a 1 in } H\text{)}$$

Motivazione. Per costruzione di H , ogni elemento della sequenza contribuisce con esattamente un bit a 1 (nel bucket della sua parte alta), e gli 0 sono solo separatori; $\#_1(H) = n$ indipendentemente dalla distribuzione nei bucket. Da qui si ricava tutto:

$$\ell = \frac{|L|}{n} \text{ (} \ell \text{ bit per elemento in } L\text{),} \quad \#_0(H) = \frac{u}{2^\ell} \text{ (numero di bucket),} \quad b = \ell + \log_2 \#_0(H).$$

È il procedimento degli hint d'esame: es. $|L| = 22$, $\#_1(H) = 11 \Rightarrow n = 11$, $\ell = 2$; con $\#_0(H) = 16$ bucket, interi originali su $2 + 4 = 6$ bit.

[8.4] Lunghezze dei codici (riferimento per gli esercizi di calcolo)

Testo dell'esercizio. (Riferimento) Numero di bit emessi dai codici unario, γ , δ , Rice e (s, c) -dense su un intero x . (vari appelli)

Detta $B(x)$ la rappresentazione binaria di $x \geq 1$ ($|B(x)| = \lfloor \log_2 x \rfloor + 1$ bit):

- **Unario** $U(x)$: x bit.
- γ : $|B(x)| - 1$ zeri seguiti da $B(x)$: $|\gamma(x)| = 2\lfloor \log_2 x \rfloor + 1$. Es.: $\gamma(9) = 000\,1001$, 7 bit.
- δ : $\gamma(|B(x)|)$ seguito da $B(x)$ intero (convenzione della dispensa): $|\delta(x)| \approx \log_2 x + 2\log_2 \log_2 x$. Es.: $x = 14$: $B(14) = 1110$, $\gamma(4) = 00\,100$, $\delta(14) = 00\,100\,1110$, **9 bit**. (Alcuni testi omettono il bit più significativo di $B(x)$: all'esame usare la convenzione della dispensa.)
- **Rice** R_k : quoziente $q = \lfloor \frac{x-1}{2^k} \rfloor$ in unario ($q+1$ bit) + resto su k bit: $|R_k(x)| = \lfloor (x-1)/2^k \rfloor + k + 1$. Es.: $R_4(83)$: $q = \lfloor 82/16 \rfloor = 5$, $5 + 4 + 1 = 10$ bit. Ottimo per interi concentrati intorno a 2^k .
- **(s, c) -dense**: alfabeto di 2^b simboli (b bit l'uno), s stopper + c continuer ($s + c = 2^b$): codeword = sequenza di continuer chiusa da uno stopper; s codeword da 1 simbolo, cs da 2, c^2s da 3, ecc. (negli esercizi: enumerare).
- **Elias–Fano**: $\approx \log_2 \frac{u}{n} + 3$ bit per intero; esatto: $n\ell + n + u/2^\ell$ ([8.1]).

20 Statistical Coding — risoluzioni

[9.1] ★ Canonical Huffman: perché, preambolo, decodifica

Testo dell'esercizio. Descrivi perché è stato introdotto il Canonical Huffman; specifica quali strutture dati tiene nel preambolo del file compresso; scrivi lo pseudocodice per decomprimere un simbolo. (09/02/24, 21/06/24, 15/07/25)

Perché si introduce. L'albero di Huffman **esplicito** ha due **difetti**: (i) **spazio**: memorizzarne la struttura nel preambolo costa $\Theta(\sigma)$ **puntatori**; (ii) **tempo**: decodificare seguendo puntatori radice→foglia fa **un salto di memoria per bit** (cache miss). Osservazione risolutiva: **dell'albero contano solo le lunghezze $L(\sigma)$** — qualunque assegnamento prefix-free con le stesse lunghezze è ottimo (Kraft). **Il canonico** assegna ai simboli di lunghezza ℓ , ordinati, **valori binari consecutivi**: la decodifica lavora con due array e confronti aritmetici (al più 2 cache miss per simbolo).

Preambolo. Per ogni lunghezza $\ell = 1, \dots, \ell_{\max}$:

- $num[\ell]$: quanti simboli hanno codeword di lunghezza ℓ ;
- $fc[\ell]$ (*first code*): valore della prima codeword di lunghezza ℓ , con la ricorrenza **dal livello più profondo verso l'alto**

$$fc[\ell_{\max}] = 0, \quad fc[\ell] = \left\lceil \frac{fc[\ell+1] + num[\ell+1]}{2} \right\rceil$$

- $symb[\ell]$: lista dei simboli di lunghezza ℓ ; la codeword del t -esimo è il numero $fc[\ell] + t - 1$ su ℓ bit.

(Sanity check con le tabelle d'esame: $num = [0, 3, 1, 2]$ dà $fc[4] = 0, fc[3] = 1, fc[2] = 1, fc[1] = 2$, cioè $FC = \{2, 1, 1, 0\}$; $fc[1] = 2$ **non è rappresentabile su 1 bit: è la sentinella** “nessuna codeword di lunghezza 1”).

Decodifica di un simbolo. Proprietà: una sequenza di bit letta come numero v su ℓ bit è una codeword completa **sse $v \geq fc[\ell]$** .

`DecodeSymbol()`:

```
v = next_bit(); len = 1
while v < fc[len]:           // non e' ancora una codeword completa
    v = 2*v + next_bit()     // aggiungi un bit
    len = len + 1
return symb[len][ v - fc[len] ] // offset dentro il livello
```

(Esempio d'esame 13/06/22: $FC = \{2, 1, 1, 0\}$, $symb = \{[], [a, e, f], [c], [d, b]\}$, input 10001: $v = 1 < 2$; $v = 10_2 = 2 \geq 1 \Rightarrow symb[2][1] = e$; poi 001: $v = 0 < 2$, $v = 0 < 1$, $v = 1 \geq 1 \Rightarrow symb[3][0] = c$.)

[9.2] Upper bound in bit della codifica aritmetica

Testo dell'esercizio. Dimostra l'upper bound in bit della codifica aritmetica, in funzione dell'entropia e della lunghezza del testo in input. (12/12/23, 09/12/24)

Teorema. Sia $T = s_1 \dots s_n$ compresso con codifica aritmetica; se $\mathcal{P}$ è la distribuzione empirica di T ed H l'entropia empirica (ordine 0), i bit emessi sono

$$\left\lceil \log_2 \frac{1}{\prod_k \mathcal{P}[s_k]} \right\rceil + 1 \leq nH + 2$$

Prova (tre passi).

(1) *Ampiezza dell'intervallo finale.* Da $[0, 1)$, ogni simbolo s_k restringe l'intervallo di un fattore $\mathcal{P}[s_k]$: dopo n simboli

$$s = \prod_{k=1}^n \mathcal{P}[s_k].$$

(2) *Bastano $\lceil \log_2 \frac{1}{s} \rceil + 1$ bit.* Si emette il **punto medio $c = lo + \frac{s}{2}$ troncato a $k = \lceil \log_2 \frac{1}{s} \rceil + 1$ bit frazionari.** Per il bound sul troncamento ([9.3]): $c - z < 2^{-k} \leq \frac{s}{2}$, quindi

$$z > c - \frac{s}{2} = lo, \quad z \leq c < lo + s:$$

il numero emesso resta dentro l'intervallo finale, e il decoder (che riproduce la stessa partizione ricorsiva di $[0, 1)$) recupera univocamente T .

(3) *Dal prodotto all'entropia.* Prendendo i logaritmi:

$$\log_2 \frac{1}{s} = \sum_{k=1}^n \log_2 \frac{1}{\mathcal{P}[s_k]} = \sum_{\sigma \in \Sigma} n_{\sigma} \log_2 \frac{1}{\mathcal{P}[\sigma]} \stackrel{\mathcal{P}[\sigma] = n_{\sigma}/n}{=} n H.$$

Quindi $k \leq \log_2 \frac{1}{s} + 2 = nH + 2$. ■

Commento. **L'overhead è di 2 bit sull'intero testo, non per simbolo:** è ciò che rende l'aritmetico quasi ottimo anche con $H \ll 1$, dove Huffman paga ≥ 1 bit/simbolo ([9.4]).

[9.3] Errore di troncamento di un numero binario < 1

Testo dell'esercizio. Dimostra un upper bound alla variazione indotta dal troncamento a k cifre di un numero < 1 rappresentato in binario come $.b_1b_2b_3 \dots$ (10/01/20)

Teorema. Sia $x = .b_1b_2b_3 \dots = \sum_{i \geq 1} b_i 2^{-i} \in [0, 1)$ e $x_k = .b_1 \dots b_k$ il troncamento alle prime k cifre. Allora

$$0 \leq x - x_k = \sum_{i > k} b_i 2^{-i} \leq \sum_{i > k} 2^{-i} = 2^{-k}$$

cioè **il troncamento a k cifre riduce il valore di al più 2^{-k}** (serie geometrica della coda; uguaglianza solo per coda tutta a 1). ■

Uso. È il lemma del passo (2) di [9.2]: troncando il punto medio a k cifre con $2^{-k} \leq s/2$ si resta dentro l'intervallo finale.

[9.4] Huffman vs Arithmetic: bit emessi e quando usare l'uno o l'altro

Testo dell'esercizio. Enuncia il numero di bit emessi da Huffman e da Arithmetic dato un testo T di lunghezza n ed entropia empirica H ; commenta quando conviene comprimere con l'uno o con l'altro, tenendo conto anche dell'efficienza in tempo. (06/03/25, 03/06/25)

Bit emessi.

$$\text{Huffman: } nH \leq |C(T)| < n(H + 1) \quad \text{Arithmetic: } |C(T)| \leq nH + 2$$

Per Huffman: la lunghezza media per simbolo dell'ottimo prefix-free soddisfa $H \leq L < H + 1$ (lower bound di Shannon; upper bound perché il codice di Shannon con lunghezze $\lceil \log \frac{1}{p_{\sigma}} \rceil$ è prefix-free e Huffman non è peggio). **Lo spreco di Huffman può avvicinarsi a 1 bit/simbolo:** ogni codeword è lunga almeno 1, anche per simboli con $\log \frac{1}{p} \approx 0$.

Quando conviene.

- **Compressione: l'aritmetico domina su distribuzioni sbilanciate ($H \ll 1$)** — es. output di MTF/RLE0 nella pipeline bzip2: Huffman emette comunque $\geq n$ bit, l'aritmetico $\approx nH + 2 \ll n$. Con H grande la differenza è trascurabile.
- **Tempo: Huffman canonico è molto più veloce** (lookup e aritmetica intera, [9.1]); l'aritmetico paga moltiplicazioni e rinormalizzazioni per simbolo.
- **Modelli adattivi/contestuali: l'aritmetico si sposa con probabilità che cambiano a ogni simbolo** (PPM); Huffman dovrebbe ricostruire il codice a ogni aggiornamento.

Sintesi: **distribuzioni skewed o modelli adattivi $\Rightarrow$ aritmetico; velocità e distribuzioni non estreme $\Rightarrow$ Huffman canonico.**

[9.5] L'aritmetico è invariante per permutazione del testo

Testo dell'esercizio. *Dimostra che due testi T_1, T_2 , l'uno permutazione dell'altro, sono compressi con lo stesso numero di bit dalla codifica aritmetica.* (23/06/25)

Prova. Dalla prova di [9.2], **il numero di bit dipende solo dall'ampiezza dell'intervallo finale:**

$$\text{bit}(T) = \left\lceil \log_2 \frac{1}{s(T)} \right\rceil + 1, \quad s(T) = \prod_{k=1}^n \mathcal{P}[t_k] = \prod_{\sigma \in \Sigma} \mathcal{P}[\sigma]^{n_\sigma(T)}.$$

L'ultimo prodotto dipende **soltanto dai conteggi $n_\sigma(T)$** , non dall'ordine dei simboli (commutatività). T_1 e T_2 , essendo l'uno permutazione dell'altro, hanno gli stessi conteggi e lo stesso modello statico $\mathcal{P}$: $s(T_1) = s(T_2)$, stesso numero di bit. ■

(I bit emessi possono *differire come stringa* — gli intervalli sono diversi — ma il loro *numero* coincide. La proprietà vale per modelli statici e count-based; è falsa per modelli contestuali di ordine ≥ 1 .)

[9.6] Perché si emette il centro (troncato) e non l'estremo sinistro

Testo dell'esercizio. *Perché la codifica aritmetica non emette l'estremo sinistro dell'intervallo finale, ma parte dall'elemento centrale e lo tronca?* (07/09/21)

L'encoder deve trasmettere abbastanza bit da identificare *un numero interno* all'intervallo finale $[lo, lo + s)$. Due problemi con lo :

- **lo può richiedere moltissimi bit (anche infiniti)**: la sua espansione binaria non ha alcun legame con l'ampiezza s ; trasmetterlo per intero vanificherebbe il bound $\approx \log_2 \frac{1}{s}$;
- **non si può troncare**: il troncamento riduce il valore ([9.3]) e qualunque numero $< lo$ è fuori dall'intervallo: il decoder sbaglierebbe.

Soluzione: partire dal **centro $c = lo + \frac{s}{2}$** e troncarlo a $k = \lceil \log_2 \frac{1}{s} \rceil + 1$ bit: per [9.3] si scende di $< 2^{-k} \leq \frac{s}{2}$, quindi si resta $> lo$, dentro l'intervallo, col minimo numero di bit legato solo a s . **Il centro è l'unico punto che tollera un errore di troncamento pari a $s/2$.**

21 Strutture dati compresse — risoluzioni

[10.1]–[10.2] ★ Rank in $O(1)$ tempo e $o(m)$ bit

Testo dell'esercizio. [10.1] Descrivi la struttura dati per supportare **Rank** su un array binario $B[1, n]$ in tempo $O(1)$ e valutarne lo spazio in bit. (16/01/23, 05/06/23, 02/11/23)
 [10.2] Dimostra la complessità in tempo e in spazio (in bit) della soluzione che implementa **Rank** in $O(1)$ e $o(m)$ bit su $B[1, m]$. (12/12/23, 10/01/25, 04/02/25)

Problema. Dato $B[1, m]$ binario statico, supportare $\text{Rank}_1(x) = \#\{i \leq x : B[i] = 1\}$ in $O(1)$ con spazio aggiuntivo $o(m)$ bit. (Da Rank_1 : $\text{Rank}_0(x) = x - \text{Rank}_1(x)$.)

Struttura a tre livelli. Parametri: $Z = (\log_2 m)^2$ (big block) e $z = \frac{1}{2} \log_2 m$ (small block).

1. *Big block*: per ognuno dei m/Z blocchi grandi, il rank **assoluto** all'inizio del blocco ($\log_2 m$ bit ciascuno).
2. *Small block*: per ognuno dei m/z blocchi piccoli, il rank **relativo** all'inizio del suo big block (valori $\leq Z$: $O(\log \log m)$ bit ciascuno).
3. *Lookup table R*: per ogni configurazione di z bit e ogni offset $o \leq z$, $R[b, o]$ = numero di 1 nei primi o bit di b (condivisa da tutti i blocchi).

Query.

$$\text{Rank}_1(x) = \text{Abs}\left[\frac{x}{Z}\right] + \text{Rel}\left[\frac{x}{z}\right] + R[B_{\text{small}(x)}, x \bmod z]$$

tre letture di array e una somma: **tempo $O(1)$ worst case** (lo small block di $\frac{1}{2} \log m$ bit si legge in $O(1)$ operazioni su parola di macchina).

Analisi dello spazio (la parte da dimostrare).

- Big block: $\frac{m}{Z} \log_2 m = \frac{m}{(\log m)^2} \log m = \frac{m}{\log m} = o(m)$ — serve $Z = (\log m)^2$ per ammortizzare i $\log m$ bit del contatore assoluto.
- Small block: $\frac{m}{z} O(\log \log m) = \frac{2m}{\log m} O(\log \log m) = O\left(\frac{m \log \log m}{\log m}\right) = o(m)$ — serve la gerarchia: il contatore relativo costa solo $\log Z = O(\log \log m)$ bit, non $\log m$.
- Lookup table: configurazioni $2^z = 2^{\frac{1}{2} \log m} = \sqrt{m}$; ciascuna con $z + 1$ offset e conteggi da $O(\log z)$ bit: $\sqrt{m} \cdot O(\log m \log \log m) = o(m)$ — serve $z = \frac{1}{2} \log m$: **l'esponente $\frac{1}{2}$ rende la tabella sub-lineare** (con blocchi di $\log m$ bit sarebbe $\Theta(m \text{ polylog})$).

Totale:

$$O\left(\frac{m \log \log m}{\log m}\right) = o(m) \text{ bit aggiuntivi,} \quad \text{query } O(1)$$

■

(Esercizi numerici — 07/09/23, 10/01/25, 23/06/25 — con parametri dati, es. $Z = 6$, $z = 2$: costruire i tre livelli e risolvere $\text{Rank}(x)$ sommando i tre contributi.)

Select (cenno). $\text{Select}_1(j)$ (posizione del j -esimo 1) si ottiene **con analogia gerarchia in $O(1)$ tempo e $o(m)$ bit**: è la primitiva usata da Elias–Fano ([8.2]) e dagli alberi succinti ([10.3]).

[10.3] Alberi binari succinti e LOUDS (riferimento operativo)

Testo dell'esercizio. (Riferimento) Codifica succinta di alberi binari in $B[1, 2n + 1]$ bit e navigazione con **Rank/Select**; **LOUDS** per alberi generali. (base degli esercizi: 11/11/20, 15/12/21, 25/07/22, 16/01/23, 10/02/23, 24/07/23, 12/12/23, 09/02/24, 10/01/25, 04/02/25, 06/03/25, 03/06/25, 23/06/25)

Codifica in $2n + 1$ bit. Si completa l'albero con le foglie esterne (NULL): ogni nodo reale ha 2 figli (reali o esterni), i nodi esterni sono $n + 1$. **Visita per livelli (BFS)** scrivendo 1 per ogni nodo reale e 0 per ogni nodo esterno: $B[1, 2n + 1]$. Etichette in un array L in ordine BFS.

Navigazione (nodo = posizione x in B con $B[x] = 1$; sia $i = \text{Rank}_1(x)$ il suo numero BFS):

$$\text{leftChild}(x) = 2i, \quad \text{rightChild}(x) = 2i + 1, \quad \text{parent}(x) = \text{Select}_1(\lfloor x/2 \rfloor)$$

(ogni nodo reale genera esattamente 2 posizioni nel livello successivo). **Test di foglia:** $B[2i] = 0$ e $B[2i + 1] = 0$. Etichetta di x : $L[\text{Rank}_1(x)]$. Tutto $O(1)$ con Rank/Select ([10.1]). È l'attrezzo per gli esercizi ricorrenti: navigare cammini, contare non-foglie, stampare etichette di foglie, verificare archi (A, B) , ecc.

LOUDS per alberi generali. Visita BFS scrivendo per ogni nodo il grado in unario ($1^{\deg 0}$), con super-radice iniziale 10: $2n + O(1)$ bit. Il j -esimo nodo BFS è il j -esimo 1; $\text{child}(j, t) = \text{Rank}_1(\text{Select}_0(j)) + t$; il padre si trova col 0 che chiude il gruppo del j -esimo 1: $O(1)$ con Rank/Select.

Nota su FM-index (possibile domanda di frontiera). La backward search mantiene l'intervallo $[first, last]$ delle righe della matrice BWT prefissate da $P[i, p]$, aggiornandolo per $c = P[i]$ con $first = C[c] + \text{Rank}_c(first - 1) + 1$, $last = C[c] + \text{Rank}_c(last)$: p passi da $O(t_{\text{Rank}})$, cioè **$O(p)$ con Rank $O(1)$** ; correttezza dall'LF-mapping della BWT.

22 Domande di progettazione — risoluzioni

[11.1] Massima somma di valori satellite (07/02/20)

Testo dell'esercizio. È data una sequenza S di n coppie $\langle \text{stringa}, \text{occ} \rangle$, dove più coppie possono riferirsi alla stessa stringa e occ è un valore satellite intero; sia s il numero di stringhe distinte. Progetta un algoritmo I/O-efficiente che calcola la stringa con massima somma dei valori satellite, valutandone la complessità I/O nei tre casi: (a) M molto più piccola di s ; (b) $M = O(s \log n)$ bit ammettendo errori in probabilità; (c) M può contenere $O(s)$ puntatori a stringhe e $O(s)$ caratteri e contatori (hint: soluzione trie-based, ma non un trie classico). (07/02/20)

Sia N la lunghezza totale in caratteri dei dati.

(a) $M \ll s$. Non si può accumulare in memoria: **si ordina**. MergeSort multi-way ([1.1]) sulle coppie con chiave la stringa: $O\left(\frac{N}{B} \log_{M/B} \frac{N}{M}\right)$ I/O. Poi una scansione: le coppie della stessa stringa sono adiacenti; si accumula la somma corrente e si mantiene il massimo: $O(N/B)$ I/O. Totale: $O(\text{sort}(N))$ I/O.

(b) $M = O(s \log n)$ bit, errori ammessi. **Contatori hashati con fingerprint**: tabella di $\Theta(s)$ celle, ognuna con un contatore ($O(\log n)$ bit) + un fingerprint di $O(\log s)$ bit (hash universale secondario). Per ogni coppia: $T[h(w)] += o$ se il fingerprint coincide. Con fingerprint di $c \log_2 s$ bit, la probabilità che una qualche coppia di stringhe distinte collida su cella e fingerprint è $\leq \binom{s}{2}/s^c$: trascurabile. Si tiene copia della miglior stringa corrente. **Una passata: $O(N/B)$ I/O**, risposta corretta con alta probabilità. (Alternativa: Count-Min sketch, con sovrastima controllata.)

(c) $O(s)$ puntatori + $O(s)$ caratteri e contatori $\Rightarrow$ **Patricia trie**. Un trie classico costerebbe troppi caratteri in memoria; **il Patricia costa $O(s)$ nodi con un carattere + uno skip per nodo** + un contatore e un puntatore su disco per foglia: esattamente il budget. Per ogni coppia $\langle w, o \rangle$: blind search di w ($O(|w|)$, 0 I/O); **verifica sulla stringa puntata dalla foglia candidata (1 I/O)**: se uguale, contatore $+= o$; se diversa, w è nuova: la si appende al file su disco e si inserisce una foglia usando la posizione di mismatch appena calcolata. Costo: tempo $O(N)$, I/O $O(n + N/B)$, memoria $O(s)$. **Rispetto ad (a) evita il sort; rispetto a (b) è esatta.**

[11.2] Finestra di k posizioni con $\geq q$ occorrenze di P (02/02/22)

Testo dell'esercizio. Dato un testo $T[1, n]$ indicizzato da un Suffix Array, progetta un algoritmo che, dati $P[1, p]$ e due interi positivi k e q , stabilisce se esiste un range di k posizioni contigue $T[i, i + k - 1]$ in cui iniziano almeno q occorrenze di P ; stampa i se esiste, altrimenti -1 . (02/02/22)

Algoritmo.

1. Ricerca di P in SA : intervallo $SA[i^*, j^*]$ delle occorrenze ([7.2]): $O(p \log n)$. Sia $occ = j^* - i^* + 1$; **se $occ < q$: stampa -1** e termina.
2. Estrai le posizioni $pos = SA[i^*..j^*]$ e **ordinale** (in SA sono in ordine lessicografico, non testuale): $O(occ \log occ)$.
3. **Sliding window**: per $t = 1, \dots, occ - q + 1$ controlla se $pos[t + q - 1] - pos[t] \leq k - 1$ (q occorrenze consecutive nel testo dentro una finestra di k posizioni). Al primo t valido stampa $i = pos[t]$; altrimenti -1 . $O(occ)$.

Correttezza. Se una finestra contiene $\geq q$ inizi, fra le posizioni ordinate ce ne sono q consecutive nella finestra, e la coppia (prima, q -esima) dista $\leq k - 1$: il test la trova; viceversa, se il test passa, la finestra che parte da $pos[t]$ contiene le q occorrenze. **Complessità.** $O(p \log n + occ \log occ)$ tempo, $O(occ)$ spazio.

[11.3] Parole $> W$ con frequenza in $[f_{\min}, f_{\max}]$ (11/11/24)

Testo dell'esercizio. È dato un insieme di coppie $\langle \text{frequenza}, \text{parola} \rangle$. Progetta una struttura dati e un algoritmo che, dati un range di frequenze $[f_{\min}, f_{\max}]$ e una parola W , restituiscono tutte le parole lessicograficamente maggiori di W con frequenza nel range; discuti complessità in tempo e spazio. (11/11/24)

È una query bidimensionale (ordine lessicografico $\times$ frequenza). Soluzione nello spirito del corso: **Treap usato come priority search tree**: chiave = parola (ordine lessicografico), priorità = frequenza a MAX-heap (radice = frequenza massima). Le priorità non sono casuali ma dati del problema: la struttura resta valida (BST $\times$ heap), si perde solo la garanzia probabilistica di bilanciamento (costruzione una tantum $O(n \log n)$).

Query ($W, f_{\min}, f_{\max}$): visita ricorsiva con **doppia potatura**:

Report(v):

```
if v = NIL: return
if freq(v) < f_min: return // MAX-heap: tutto il sottoalbero < f_min
if key(v) > W:
    if freq(v) <= f_max: output (key(v), freq(v))
    Report(left(v)); Report(right(v))
else:
    // key(v) <= W: a sinistra tutto <= W
    Report(right(v))
```

Correttezza delle potature: (i) **per il MAX-heap, se $freq(v) < f_{\min}$ nessun discendente è in range**; (ii) **per il BST, se $key(v) \leq W$ tutto il sottoalbero sinistro ha chiavi $\leq W$** .

Complessità. Spazio $O(n)$. Query $O(h + occ')$ con h altezza e $occ' = \#\{\text{parole} > W \text{ con } freq \geq f_{\min}\}$: per la query three-sided ($> W$) $\times$ $[f_{\min}, \infty)$ è l'output-sensitive classico $O(h + occ)$; il vincolo $\leq f_{\max}$ non aiuta la potatura (i nodi troppo frequenti vanno comunque attraversati). *Alternativa con garanzie*: range tree con query $O(\log^2 n + occ)$ e spazio $O(n \log n)$ — da citare come trade-off.